

Bachelor Thesis (173312)
Angewandte Informatik (SPO1a)

Einsatz von Künstlicher Intelligenz zur Testfallgenerierung

Marvin Müller*

25. Februar 2025

Eingereicht bei Prof. Dr. rer. nat. Nicole Ondrusch

*212117, mmueller3@stud.hs-heilbronn.de

Inhaltsverzeichnis

Abkürzungsverzeichnis	IV
Abbildungsverzeichnis	V
Gender-Hinweis	VI
Abstract	VII
Zusammenfassung	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Forschungsfrage	2
1.3 Ziel der Arbeit	3
1.4 Vorgehensweise	3
2 Grundlagen und Hintergrund	4
2.1 KI	4
2.1.1 Allgemein	4
2.1.2 Bereiche der KI	6
2.2 Large Language Model (LLM)	8
2.2.1 Entwicklungen im Bereich LLM	9
2.3 Prompting	9
2.3.1 Prompt Engineering und Prompt-Techniken	11
2.4 Retrieval Augmented Generation (RAG)	11
2.4.1 Einsatzmöglichkeiten von RAG	13
2.5 Testfall	13
2.5.1 Aufbau eines Testfalls	14
2.6 Anforderungsspezifikation	15
2.6.1 Bedeutung der Anforderungsspezifikation für die Testfallgenerierung	16
3 Related Work und aktuelle Forschung	17
4 Methodik	21
4.1 Verwendetes LLM	21
4.2 Testdaten	22
4.3 Prompt und verwendete Methoden	24
4.4 Qualitätskriterien	25
4.4.1 Testfallstruktur	25
4.4.2 Factual Correctness	26
4.4.3 Coverage (Deutsch: Abdeckung)	28
5 Ergebnisse	31
6 Diskussion	36
7 Fazit und Ausblick	38

Literatur	39
Eidesstattliche Erklärung	43
Anhang	44

Abkürzungsverzeichnis

AI (KI): Artificial Intelligence (Deutsch: Künstliche Intelligenz)

GUI: Graphical User Interface (Deutsch: Grafische Benutzeroberfläche)

IEC: International Electrotechnical Commission

IEEE: Institute of Electrical and Electronics Engineers

ISO: International Organization for Standardization

ISTQB: International Software Testing Qualifications Board

LLM: Large Language Model (Deutsch: Großes Sprachmodell)

NLP: Natural Language Processing (Deutsch: Verarbeitung natürlicher Sprache)

RAG: Retrieval Augmented Generation

Abbildungsverzeichnis

2.1	Die Grafik von Hecker et al. (2018) zeigt die Interaktion eines KI-Systems mit seiner Umwelt	5
2.2	Die Grafik von Mocko und Bitton-Bailey (o. D.) zeigt die Beziehung zwischen KI, maschinellem Lernen, neuronalen Netzen und Deep Learning . .	6
2.3	Die Grafik von Dymatrix (2018) zeigt ein einfaches Neuronales Netz und ein Deep Learning Netz	7
2.4	Die Grafik von Hodiak (2024) zeigt LLM und NLP und verschiedene Anwendungsfälle der Technologien	8
2.5	Ein Prompt in GPT-4o von OpenAI (o. D.) in der Benutzeroberfläche . . .	10
2.6	Die Grafik von Honroth et al. (2024) zeigt, wie Retrieval Augmented Generation (RAG) im Detail funktioniert	12
2.7	Die Grafik von Enderli (2019) zeigt einen Testfall im SAP Solution Manager	14
2.8	Die Grafik von Jama Software (o. D.) zeigt die Eigenschaften von funktionalen vs. nicht funktionalen Anforderungen	15
4.1	Ein von Pavankumar21github (2022) erstellter Testfall, welcher die Erstellung eines Accounts in OpenCart (o. D.) anhand der oben gezeigten Anforderungsspezifikation testet und als Benchmark für die von GPT-4o erstellten Testfälle dient	23
4.2	Ein Prompt an GPT-4o	24
4.3	Ein von GPT-4o erstellter Testfall, welcher nicht der Testfallstruktur entspricht	25
4.4	Ein von GPT-4o erstellter Testfall, welcher der Testfallstruktur entspricht und nur faktisch korrekte Testschritte besitzt	28
4.5	Vergleich dreier Testfälle, die eine unterschiedliche Qualität der Qualitätskriterien aufweisen.	30
5.1	Factuall Correctness pro Testszenario	33
5.2	Abdeckung pro Testszenario	35

Gender-Hinweis

In dieser Bachelorarbeit wird aus Gründen der besseren Lesbarkeit das generische Maskulinum verwendet. Dabei gilt die männliche Sprachform für alle Geschlechter.

Abstract

This bachelor thesis investigates the use of the LLM GPT-4o from OpenAI (o. D.) in the creation of requirement test cases, taking into account the requirement specifications provided to it. The aim of the work and the evaluation of the results is to find out whether GPT-4o is able to generate requirement test cases that are suitable for practical use in test case generation with regard to their coverage, factual correctness and test case structure according to ISO/IEC/IEEE 29119-1:2022-01 (2022). A total of 128 test cases were generated for ten different test scenarios. In order to check factual correctness, these are checked against the information on a sample website. Manually created test cases from an open source project that test the same test scenarios serve as a benchmark for coverage..

The test cases generated by GPT-4o achieved a value of 100 % for the test case structure, an average value of 78.1 % for factual correctness and a value of 68 % for the coverage rate. The results of this bachelor thesis show that the use of artificial intelligence for test case generation has great potential and that the findings from this thesis can be used in future works.

Keywords: AI, LLM, RAG, Prompting, Test Case Generation

Zusammenfassung

Diese Bachelorarbeit erforscht den Einsatz des LLM GPT-4o von OpenAI (o. D.) bei der Erstellung von Anforderungstestfällen unter Berücksichtigung ihrer zur Verfügung gestellten Anforderungsspezifikationen. Ziel der Arbeit und der Auswertung der Ergebnisse ist es herauszufinden, ob GPT-4o dazu in der Lage ist, Anforderungstestfälle zu erzeugen, welche in Bezug auf ihre Abdeckung, Factual Correctness und Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) für den praktischen Einsatz in der Testfallerstellung geeignet sind. Dazu wurden für zehn unterschiedliche Testszenarien insgesamt 128 Testfälle generiert. Um die Factual Correctness zu überprüfen, werden diese mit den Informationen auf einer Beispiels Website überprüft. Manuell erstellte Testfälle eines Open Source Projektes, welche die gleichen Testszenarien testen, dienen als Benchmark zur Abdeckung.

Dabei erzielten die von GPT-4o generierten Testfälle bei der Testfallstruktur einen Wert von 100 %, bei der Factual Correctness einen Durchschnittswert von 78,1 % und bei der Abdeckungsrate einen Wert von 68 %. Die Ergebnisse dieser Bachelorarbeit zeigen, dass der Einsatz von Künstlicher Intelligenz zur Testfallgenerierung ein großes Potenzial besitzt, die Erkenntnisse aus dieser Arbeit können in zukünftigen Werken weiter genutzt werden.

Stichwörter: KI, LLM, RAG, Prompting, Testfallgenerierung

1 Einleitung

Kapitel 1 widmet sich der Darstellung der Motivation, die der Bachelorarbeit zugrunde liegt und erläutert die Zielsetzung. Dabei wird verdeutlicht, warum das Thema Einsatz von Künstlicher Intelligenz zur Testfallgenerierung relevant ist und welchen Beitrag die Arbeit dazu leistet. Darüber hinaus wird in diesem Kapitel die zentrale Forschungsfrage formuliert und die methodische Vorgehensweise in dieser Arbeit beschrieben. Diese Erläuterungen definieren den inhaltlichen und methodischen Rahmen dieser Arbeit.

1.1 Motivation

In den vergangenen Jahren hat in dem Bereich Künstliche Intelligenz (KI) eine rasante Entwicklung stattgefunden. Diese hat dabei Auswirkungen auf das private als auch auf das geschäftliche Leben. Der Artikel des Branchenverbandes der deutschen Informations- und Telekommunikationsbranche - Bitkom - stellt eine von ihm durchgeführte Studie vor. Laut dieser Studie hat sich der Anteil der deutschen Unternehmen, welche KI nutzen, von 9 % im Jahr 2022 auf 15 % im Jahr 2023 nahezu verdoppelt. Dabei sehen 68 % der Befragten KI als Chance. (Streim und Hecker, 2023)

In einer Pressemitteilung vom 25. November 2024 gibt das Statistische Bundesamt bekannt, dass bereits 20 % der in Deutschland ansässigen Unternehmen KI verwenden. Dabei sind die drei häufigsten Einsatzbereiche für KI Textmining - Analyse geschriebener Sprache mit 48 %, Spracherkennung mit 47 % und die Erzeugung von natürlicher Sprache mit 34 %. Dies zeigt eine rasante Steigerung der Nutzung von KI alleine in den letzten drei Jahren und es lässt sich erkennen, dass KI bereits vielfältig einsetzbar und nutzbar ist. Gerade in der Informatik und in deren Teilgebiet der Softwareentwicklung trifft dies zu. (Statistisches Bundesamt, 2024)

In dieser Arbeit stehen KI-Sprachmodelle im Vordergrund. Diese sogenannten Large Language Models (LLM) bieten sich dafür an, redundante und sich ähnliche Aufgaben für den Anwender zu erledigen. LLM werden deshalb bereits in verschiedensten Bereichen der Textgenerierung verwendet. Zu nennen sind dabei Übersetzung, Texterstellung, Textzusammenfassung, Kategorisierung und Klassifizierung. Eine Anwendungsmöglichkeit der Textgenerierung ist deshalb die Testfallgenerierung durch die KI. Das Erstellen von Testfällen mithilfe von KI kann erhebliche Zeit- und Kosteneinsparungen für Unternehmen und Tester ermöglichen. (Kerner, 2024)

Ein Testfall hat eine Struktur, welche definierten Regeln entspricht. In der Softwareentwicklung, aber auch allgemein in fast allen beruflichen und wissenschaftlichen Bereichen, werden mithilfe von Testfällen Szenarien getestet. Somit ist die Testfallgenerierung durch KI nahezu universell einsetzbar. Es gibt bereits Arbeiten, welche sich mit dem Thema „KI erzeugte Testfälle“ KI erzeugte Testfälle beschäftigt haben. Zu nennen sind dabei die Werke von (Bhandari et al., 2024; S. Gu et al., 2024; Khan et al., 2024; Kirinuki & Tanno, 2024; Z. Liu et al., 2024; Ouédraogo et al., 2024). Vorteile von KI generierten Testfällen sind dabei deren gesteigerte Qualität und Kosten- und Zeiteinsparungen. (Bozic, 2022; Weingartz und Suleymanov, 2024)

Wie bei allen neuen Technologien ist es essenziell, sich auch der damit verbundenen Risiken bewusst zu sein. Ein Nachteil der KI ist zum Beispiel das Phänomen der Halluzination. Dieser Begriff beschreibt das Phänomen, dass eine KI Antwort nicht mit dem gegebenen Input übereinstimmt und faktisch nicht richtig ist. (Siebert, 2024)

Ebenfalls ist der Umgang mit Daten nicht sicher. Das Problem erstreckt sich sowohl auf unternehmensinterne und als auch auf persönliche Daten. Es gibt keine Transparenz darüber, wie diese behandelt werden, wenn sie einem LLM zur Verfügung gestellt werden. Persönliche Daten können missbraucht werden und LLM können mit unternehmensinternen Daten antrainiert werden, welche die Konkurrenz dann nutzen kann. Eine Maßnahme um die genannten Risiken zu minimieren, ist die Anonymisierung persönlicher und sensibler Daten, bevor diese dem LLM zur Verfügung gestellt werden. (Möllers, 2024)

Da die im letzten Absatz beschriebene Lösung mit Zeit und Aufwand verbunden ist, bieten sich andere Methoden an. Zum Beispiel gibt es Modelle, welche Retrieval Augmented Generation (RAG) verwenden. Diese eignen sich für die Aufgabe der Testfallgenerierung. Die Technik RAG beschreibt den Vorgang, dass das Wissen, welches von der LLM benötigt wird, nicht aus dem Prompt - der Eingabe - kommen muss. Informationen aus Dateien, die dem LLM zur Verfügung gestellt werden, können ebenfalls verwendet werden. (Honroth et al., 2024)

1.2 Forschungsfrage

Die folgende Forschungsfrage, die aus der zugrunde liegenden Motivation abgeleitet wurde, bildet die Grundlage für die Durchführung der Arbeit:

Ist es möglich, dass das LLM GPT-4o von OpenAI (o. D.) dazu in der Lage ist, aus Anforderungsspezifikationen und Screenshots mit Website-Informationen, Anforderungstestfälle zu erzeugen, die in Bezug auf messbare Kriterien für den praktischen Einsatz in der Testfallerstellung geeignet sind?

Diese Frage zielt darauf ab, herauszufinden, ob ein frei verfügbares LLM wie GPT-4o in der Lage ist, Testfälle zu generieren und ob somit Nutzer wertvolle Zeit und Ressourcen sparen können, wenn die KI redundante Aufgaben wie das Erstellen von Anwendungstestfällen übernimmt. Die Ergebnisse müssen dann überprüft werden. Um diese Frage zu beantworten, werden Auswertungsmetriken entwickelt, anhand derer die von GPT-4o erstellten Testfälle ausgewertet werden können. Außerdem wird die gängige Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) definiert und die faktische Korrektheit der Testfälle überprüft. Dabei dienen manuell erstellte Testfälle, welche aus der gleichen Anforderungsspezifikation erstellt wurden als Benchmark, also als Vergleichsmaßstab.

1.3 Ziel der Arbeit

Ziel der Arbeit ist es, die Grundlagen und Anwendungsmöglichkeiten und -verfahren zur Testfallgenerierung durch KI zu erforschen, durchzuführen und zu evaluieren. Diese Bachelorarbeit soll eine Vorgehensweise entwickeln, die sich für den Einsatz von KI für die Generierung von Anforderungstestfällen aus der KI zur Verfügung gestellten Anforderungsspezifikationen eignet. Diese stützt sich dabei auf bereits bestehender Vorgehensweisen und Modelle. Durch die theoretische und praktische Aufarbeitung und Erprobung bestehender Vorgehensweisen und Modelle, soll sie dabei einen Beitrag für Nutzer leisten, die Effizienz und Qualität der mit Hilfe von KI erstellten Testfälle zu verbessern.

1.4 Vorgehensweise

Das Vorgehen in der vorliegenden Arbeit erfolgt in mehreren Abschnitten. Zunächst wird eine Literaturrecherche durchgeführt. Dabei liegt der Fokus auf dem Recherchieren von bereits vorhandener Vorgehensweisen und Modellen im Bezug auf Textgenerierung im Allgemeinen und Testfallgenerierung im Besonderen. Es ist ebenfalls notwendig, sich über Begriffe und Techniken, welche während der Umsetzung der Arbeit notwendig sind, tiefer zu befassen. Wichtig ist dabei, eine festgelegte Struktur zu finden oder zu definieren, wie ein Testfall und eine Anforderungsspezifikation aufgebaut sind, welche Merkmale und welche Kriterien sie erfüllen müssen. Es wird recherchiert, um was es sich handelt und warum sich GPT-4o von OpenAI (o. D.) als LLM für die Testfallgenerierung eignet. Das Ermitteln geeigneter Prompting Methoden sowie der Methode RAG ist ebenfalls erforderlich. Das Ziel ist dabei die Untersuchung und Entdeckung bereits vorhandener Anwendungsmöglichkeiten, Vorgehensweisen und Modellen zur GPT-4o gesteuerten Testfallgenerierung, anhand derer die Testfallgenerierung in dieser Arbeit ausgeführt werden kann.

Nachdem das Vorgehen ermittelt wurde und passende Testdaten, welche sich in Anforderungsspezifikationen und aus diesen manuell erstellte Anforderungstestfälle gliedern, gefunden wurden, beginnt die praktische Umsetzung der Arbeit. Dabei liegt die Durchführung, Dokumentation und Auswertung der von GPT-4o erstellten Anforderungstestfälle im Vordergrund.

Zum Schluss werden die durch GPT-4o erstellten Anforderungstestfälle auf ihre Abdeckung, Factual Correctness und Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) ausgewertet. Die faktische Korrektheit wird überprüft, indem die von GPT-4o generierten Testfälle mit manuell erstellten Anforderungstestfällen verglichen und auf ihre Korrektheit eingeordnet werden. Die Ergebnisse werden analysiert und hinsichtlich ihrer Abdeckung, Factual Correctness und Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) ausgewertet. Dabei werden für die Auswertung Metriken verwendet, die in Kapitel 4.4 näher erläutert werden.

2 Grundlagen und Hintergrund

Das Kapitel 2 Grundlagen und Hintergrund widmet sich den wissenschaftlichen Grundlagen und klärt über Begriffe und Methoden auf, die für das Verstehen und das Durchführen der Arbeit essenziell sind. In diesem Kapitel wird erst der Begriff KI erklärt und danach erläutert, welche Teilbereiche es gibt. Anschließend wird der Begriff LLM erläutert, da das LLM GPT-4o von OpenAI (o. D.) für das Erstellen der Testfälle verwendet wird. Danach werden Prompting und RAG beschrieben. Wichtig ist ebenfalls zu klären, wie ein Testfall aufgebaut und was eine Anforderungsspezifikation ist, um die von GPT-4o erstellten Anforderungstestfälle nach dem in Kapitel 4.4.1 aufgestellten Qualitätskriterium „Testfallstruktur“ auswerten zu können.

2.1 KI

In dieser Arbeit wird KI für die Generierung von Anforderungstestfällen verwendet. Da KI ein weitverbreiteter Begriff ist und es viele Unterbereiche der KI gibt, ist es wichtig, dass hier erst ein grober Überblick über den Begriff KI verschafft wird. Danach werden einzelne Unterbereiche der KI vorgestellt und beschrieben.

2.1.1 Allgemein

Der Begriff KI beziehungsweise im Englischen Artificial Intelligence (AI) beschreibt ein Teilgebiet der Informatik. Dabei gibt es zwei Typen von KI. Erstens die schwache KI, welche auch als enge KI bezeichnet wird. Diese Art der KI kann nur einzelne und beschränkte Aufgaben lösen. Der Begriff KI wurde bereits im Jahr 1956 als Konzept vorgeschlagen. Seitdem entwickelte sie sich immer weiter. Im Jahr 1997 verlor der damalige Schachweltmeister Garry Kasparov gegen den Schachcomputer Deep Blue von IBM und im Jahr 2016 besiegte der Computer AlphaGo von Google Lee Se-dol, einen Go-Meister aus Südkorea. Diese Erfolge zeigen die Entwicklung der KI, sind jedoch auf die schwache KI beschränkt. (Hecker et al., 2018)

Als Zweites gibt es das Konzept der starken KI, welche auch als allgemeine KI bekannt ist. Folgende Grafik zeigt, wie eine starke KI ihre Umgebung wahrnimmt.

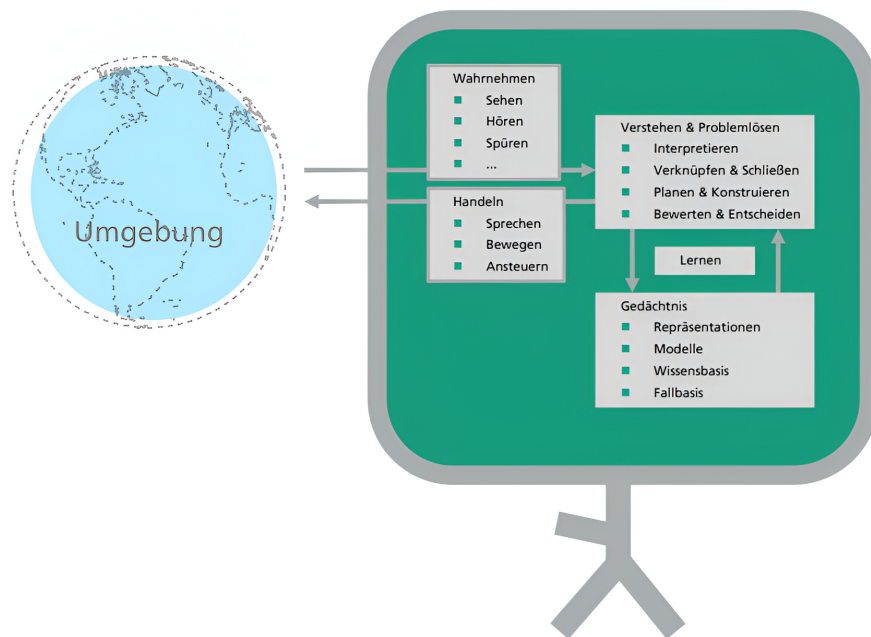


Abbildung 2.1: Die Grafik von Hecker et al. (2018) zeigt die Interaktion eines KI-Systems mit seiner Umwelt

Das Konzept hinter dieser KI besteht darin, dass Maschinen Fähigkeiten verwenden beziehungsweise imitieren können, die menschlich sind. Laut Hecker et al. (2018) bilden sie kognitive Fähigkeiten ab. Es scheint dabei, dass die Maschinen sich selbst und ihre Umgebung wahrnehmen und interpretieren können. Ihre Wahrnehmung und ihre Interpretation beruht sich dabei auf Daten, Fakten, konkreten Modellen und Regeln, welche ihr nächstes Handeln bestimmen. Während des Nutzens lernen die Maschinen dabei kontinuierlich weiter, so als würden sie ein Gedächtnis besitzen. (Bhatt et al., 2020; Zhu et al., 2021)

Hecker et al. (2018) nennt noch eine dritte Form der KI, die sogenannte Superintelligenz, diese wird es jedoch laut Hecker et al. (2018) in nächster Zeit nicht geben und somit wird in dieser Arbeit nicht auf diese eingegangen.

2.1.2 Bereiche der KI

Nachdem erklärt wurde, um was es sich bei KI handelt, ist es nun wichtig zu erklären, welche Bereiche und Teilgebiete es von KI gibt. Zu nennen sind dabei maschinelles Lernen, Neuronale Netze und Deep Learning.

Folgende Grafik zeigt anschaulich die einzelnen Teilgebiete der KI.

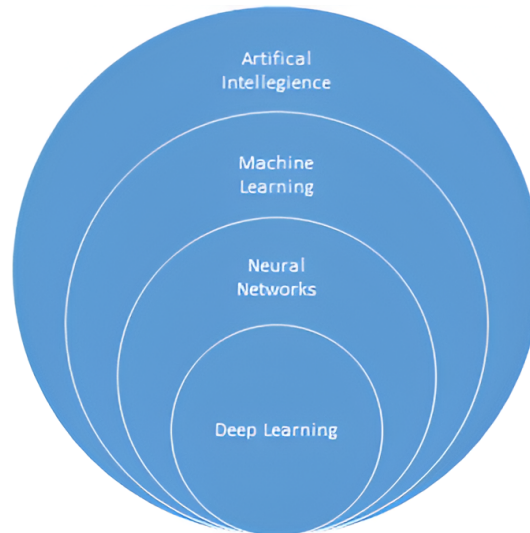


Abbildung 2.2: Die Grafik von Mocko und Bitton-Bailey (o. D.) zeigt die Beziehung zwischen KI, maschinellem Lernen, neuronalen Netzen und Deep Learning

KI wird bereits in Kapitel 2.1 beschrieben. Die Grafik ist dabei so zu verstehen, dass der größere Kreis die jeweils kleineren Kreise umfasst, das heißt maschinelles Lernen ist ein Teil von KI, neuronale Netze sind ein Teilgebiet von KI und von maschinellem Lernen und Deep Learning ist ein Teilgebiet von KI, von maschinellem Lernen und von Neuronalen Netzen. (Bhatt et al., 2020; Zhu et al., 2021)

Im Folgenden werden nun einige wichtige Bereiche der KI aufsteigend vom kleinsten zum größten Teilbereich erläutert.

Deep Learning:

Deep Learning ist ein Teilgebiet der neuronalen Netze. Sie sind dadurch gekennzeichnet, dass sie ein „Hidden Layer“ besitzen, welches aus mehr als einer Neuronenschicht besteht. In diesem werden die Informationen neu gewichtet (Bhatt et al., 2020); (Zhu et al., 2021). Folgende Grafik zeigt anschaulich ein Neuronales Netz und ein Deep Learning Netz.

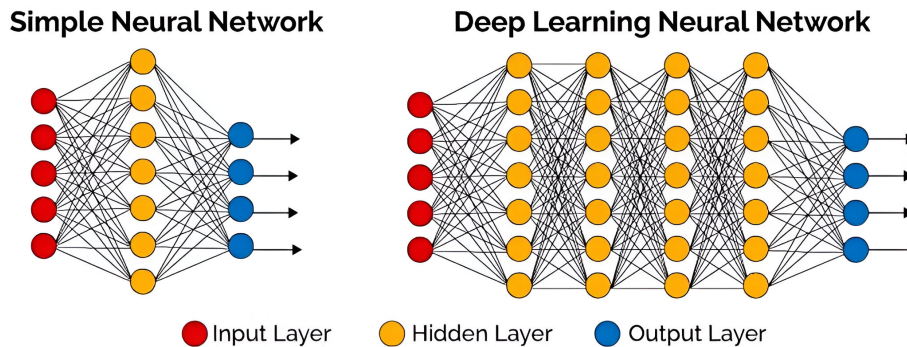


Abbildung 2.3: Die Grafik von Dymatrix (2018) zeigt ein einfaches Neuronales Netz und ein Deep Learning Netz

Deep Learning Netze können dabei Millionen Neuronenschichten, im Hidden Layer, beinhalten und dadurch immer komplexere Aufgaben lösen. (Bhatt et al., 2020; Zhu et al., 2021)

Weitere wichtige KI-Bereiche wie LLM, Prompting und RAG werden in den Kapiteln 2.2, 2.3 und in Kapitel 2.4 erläutert, da diese eine wichtige Rolle in dieser Arbeit spielen.

Neuronale Netze:

Bei neuronalen Netzen handelt es sich um ein Teilgebiet des maschinellen Lernens. Idee der neuronalen Netze ist es, menschliche Neuronen nachzubilden und somit menschliches Denken nachzuahmen. Ein neuronales Netz besteht dabei aus Knoten, die aneinandergereiht sind. Sie haben eine Eingabeschicht, ein Hidden Layer, welches aus einer Neuronenschicht besteht und eine Ausgabeschicht, die miteinander vernetzt sind. Sie trainieren in Schleifen und Durchgängen und werden durch Wiederholung besser und schneller. (Bhatt et al., 2020; Zhu et al., 2021)

Maschinelles Lernen:

Maschinelles Lernen, auf Englisch Machine Learning, ist ein Teilgebiet der KI. Es beschreibt das Verfahren, in welchem Maschinen durch das Wiederholen von Aufgaben lernen. Dabei gibt es Algorithmen, welche mit Daten antrainiert werden und der Lernprozess ist eine Schleife, in welcher diese Algorithmen wiederholt werden. Dadurch lernt die Maschine die Aufgabe zu lösen. Dies geschieht, in dem die Maschine Feedback erhält und so lernt, wie sie die Aufgabe richtig löst. Es wird dabei kein Lösungsweg vorgegeben wie bei herkömmlichen Algorithmen, sondern die Maschine lernt durch Versuch und Irrtum oder auf Englisch Trial and Error. (Bhatt et al., 2020; Zhu et al., 2021)

2.2 Large Language Model (LLM)

In dieser Arbeit wird das LLM GPT-4o von OpenAI (o. D.) verwendet, um Testfälle aus Anforderungsspezifikationen zu generieren. Der Begriff LLM kann als großes Sprachmodell übersetzt werden. Über die Eigenschaften und Entwicklungen von LLM klärt dieses Kapitel auf. Dies ist wichtig, um zu verstehen, weshalb in dieser Arbeit ein LLM wie GPT-4o genutzt wird.

Diese LLMs stellen eine Kombination aus zwei Bereichen der KI dar. Sie sind ein Teilgebiet von Natural Language Processing (NLP). Dieser Prozess beschreibt die Möglichkeit der Verarbeitung natürlicher Sprache. Dabei wird dieser mithilfe der Trainingsmöglichkeit, welche Deep Learning bietet, kombiniert. (Chang et al., 2023; Kaddour et al., 2023; Naveed et al., 2024)

Folgende Grafik zeigt anschaulich, in welchem Gebiet LLM innerhalb der KI angesiedelt ist:

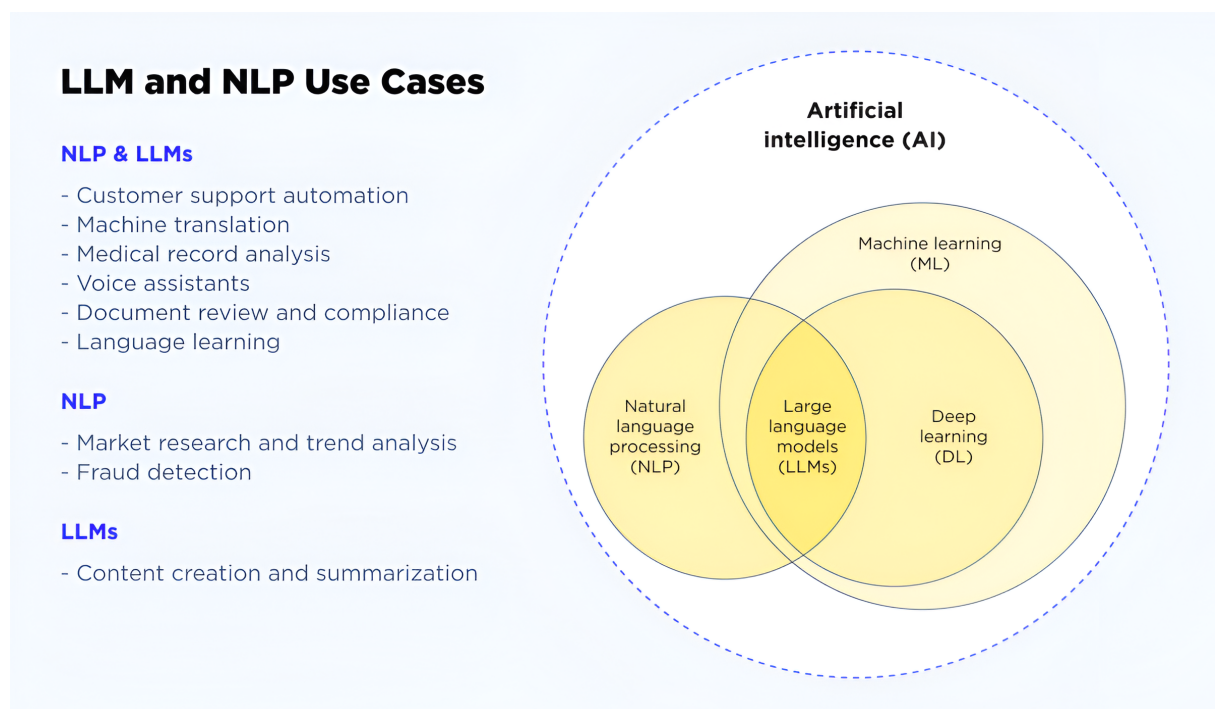


Abbildung 2.4: Die Grafik von Hodiak (2024) zeigt LLM und NLP und verschiedene Anwendungsfälle der Technologien

Dabei können diese Modelle durch das Training und der Verfügbarkeit von großen Datenmengen natürliche Sprache nachahmen. Ein bekanntes Beispiel für LLM sind Chatbots wie GPT-4o. Diese LLM können dabei auf verschiedenste Aufgaben entwickelt und trainiert werden und sind somit vielseitig einsetzbar. Häufige Aufgaben sind laut Kerner (2024) die bereits erwähnten Chatbots, Textgenerierung und auch die Forschung und Entwicklung. Dabei können LLM mit vielen Sprachen arbeiten, welche sie nahezu universell einsetzbar machen. (Chang et al., 2023; Kaddour et al., 2023; Naveed et al., 2024)

2.2.1 Entwicklungen im Bereich LLM

Aufgrund des Erfolgs von LLMs, ihren Möglichkeiten und ihren vielseitigen Einsatzmöglichkeiten sind sie dynamisch. Neue Modelle können selbst entwickelt werden. Vorhandene Modelle können auch antrainiert werden, was als Finetuning bezeichnet wird. Ein Werk, welches sich ausführlich mit dem Thema LLM erstellen und finetunen beschäftigt, ist Géron (2019). Das Erzeugen und das Finetuning von LLM benötigt jedoch Zeit, Ressourcen und eine Menge an Daten. (Géron, 2019)

Deshalb ist es effizienter und präziser, sich bereits bestehende Modelle anzusehen, welche es bereits in großer Anzahl gibt. Eine bekannte Open Source Plattform, in welcher nach LLM gesucht und diese heruntergeladen werden können, ist die Website Hugging Face (o. D.). Sie bietet eine große Auswahl von LLM an. Während am 16.01.2025 auf ihr 1.295.659 verschiedene Modelle angeboten wurden, wurden etwas mehr als zwei Wochen später am 04.02.2025, bereits 1.378.895 verschiedene Modelle angeboten. Dies zeigt die enorme Vielfalt und die rapide Entwicklung im Bereich LLM, in welcher im Minutentakt neue Modelle hinzukommen.

2.3 Prompting

Die Eingabe bzw. der Prompt ist ein wesentlicher Bestandteil zum Nutzen von KI. Jede Eingabe ist ein Prompt und die Qualität des Prompts kann über die Qualität der Antwort, welche das LLM liefert, entscheiden. Deshalb ist es wichtig, diesen Begriff und die Vorgehensweise, nach welcher in dieser Arbeit vorgegangen wird, zu erklären. Der englische Begriff Prompting beziehungsweise Prompt kann als Eingabeaufforderung übersetzt und verstanden werden. Mithilfe eines Prompts kann der Nutzer der LLM eine Aufgabe oder eine Mitteilung liefern, welche diese verarbeiten und dann lösen kann. Prompting ist ein entscheidendes Element, um mit LLM zu interagieren. (J. Gu et al., 2023)

Folgender Screenshot zeigt einen Prompt und die Antwort der LLM in diesem Fall von GPT-4o von OpenAI (o. D.):

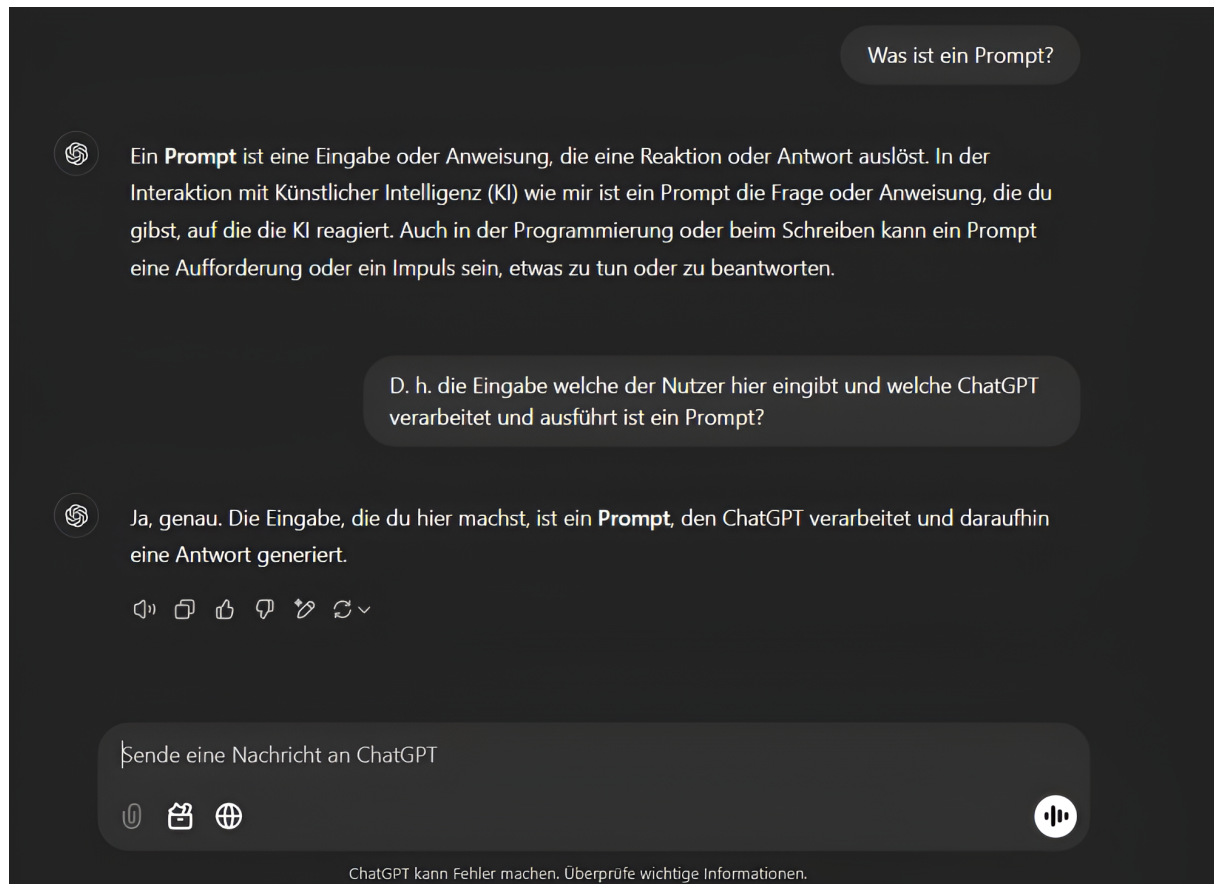


Abbildung 2.5: Ein Prompt in GPT-4o von OpenAI (o. D.) in der Benutzeroberfläche

Prompts können dabei einfache Sprachaufforderungen sein, wie anhand des in der Grafik gezeigten Beispiels. Jedoch können diese auch komplexer sein und für vielseitige Aufgaben unterschiedlich genutzt werden. Dabei gibt es den Begriff Prompt Engineering und verschiedene Prompt-Techniken, welche in Kapitel 2.3.1 erklärt werden. (J. Gu et al., 2023; P. Liu et al., 2023)

2.3.1 Prompt Engineering und Prompt-Techniken

Unter dem Begriff Prompt Engineering wird das gezielte Nutzen von Prompts, um bessere Ergebnisse zu erhalten, verstanden. Dabei wird systematisch vorgegangen, um ohne das Finetuning von LLM konkretere und verbesserte Lösungen von diesen zu erhalten. Es gibt viele Prompt-Techniken. Eine Prompt-Technik, welche sich flexibel an unterschiedlichste Aufgaben anpassen kann, ist das sogenannte Soft Prompting. In diesem werden Parameter, welche lernen können, direkt in die Eingabevektoren des Modells eingebettet. Dadurch müssen die Prompts nicht als Text eingegeben werden. Wichtig ist zu verstehen, dass Prompting flexibel ist und es viele Möglichkeiten gibt. In dieser Arbeit wird eine spezielle Technik verwendet, welche in Kapitel 2.4 erklärt wird. (J. Gu et al., 2023; P. Liu et al., 2023)

2.4 Retrieval Augmented Generation (RAG)

Bereits in Kapitel 1.1 wird RAG erwähnt. Sie ermöglicht es, einem LLM Dokumente zur Verfügung zu stellen, in welcher sich Wissen befindet. In dieser Arbeit kann dem LLM per RAG eine Anforderungsspezifikation zur Verfügung gestellt werden, in welcher die Anforderungen zu finden sind, aus welchen das LLM die Anforderungstestfälle generieren soll. Deshalb ist RAG für diese Arbeit von besonderer Wichtigkeit. Dabei werden die in den Dokumenten enthaltenen Informationen für das LLM nutzbar gemacht. Die Dokumente und Anfragen werden dabei in kleine Abschnitte, welche als Chunks bezeichnet werden, aufgeteilt. RAG lässt sich dabei in vielen LLM anwenden und optimiert diese, ohne dass diese mit viel Aufwand trainiert werden müssen. (Gao et al., 2024; Wu et al., 2024)

Folgende Grafik zeigt anschaulich, wie RAG funktioniert:

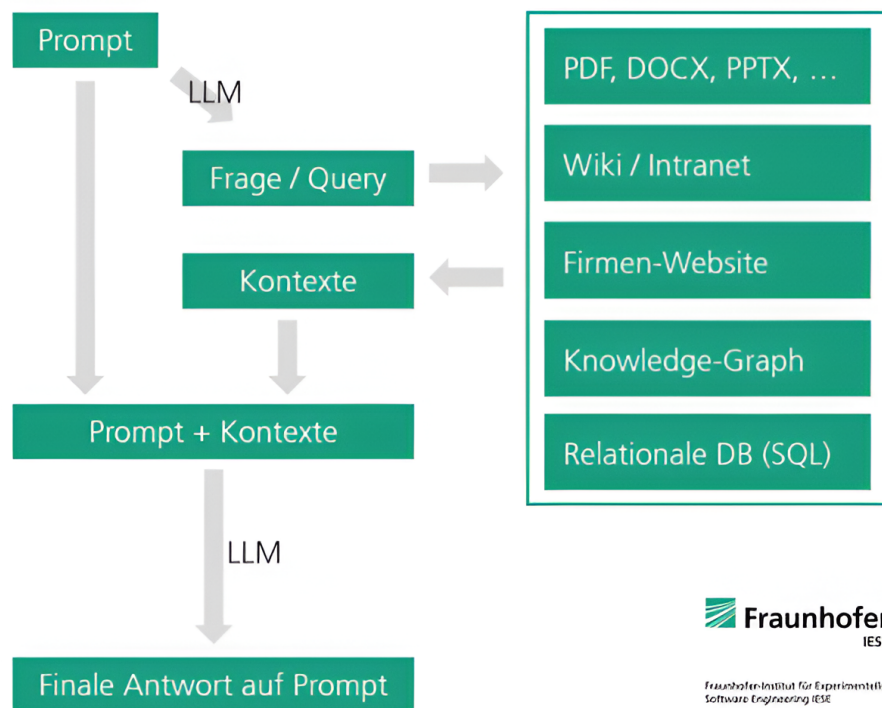


Abbildung 2.6: Die Grafik von Honroth et al. (2024) zeigt, wie Retrieval Augmented Generation (RAG) im Detail funktioniert

Im Folgenden werden die einzelnen Schritte von RAG erklärt, dabei gibt es laut Schmid und Sommer (2024) und laut Miesle (2023) fünf Schritte. In dieser Arbeit werden die Begriffe so verwendet, wie sie Schmid und Sommer (2024) verwendet haben, da es sich um ein Werk auf Deutsch handelt und diese Arbeit ebenfalls auf Deutsch ist:

1. Indexierung:

Die Indexierung beschreibt den Prozess, wie die Daten aus einem Dokument in kürzere Abschnitte, den Chunks, aufgeteilt werden, um die Daten für das LLM verwertbar zu machen. Die Chunks werden maschinenlesbar gemacht und in einer Vektordatenbank gespeichert, aus welcher sie nun für das LLM zugreifbar sind. Sie werden vektorisiert.

2. Benutzereingabe:

Die Benutzereingabe erfolgt als Prompt in natürlicher Sprache, welche sich auf die im Dokument befindlichen Daten beziehen.

3. Retrieval:

Nachdem die Maschine die Eingabe erhalten hat und diese ebenfalls vektorisiert hat, wird mit der Methode des nächsten Nachbarn die höchste Ähnlichkeit zwischen der Eingabe und der in der Vektordatenbank gespeicherten Chunks priorisiert.

4. Augmentation:

Das LLM wird diese Kombination aus Prompt des Benutzers und dem Kontext zu Verfügung gestellt.

5. Generation:

Es wird nun eine Antwort generiert, welche sowohl die erhaltenen Informationen als auch das Training des jeweiligen LLM kombiniert.

RAG ist vielfältig einsetzbar und eignet sich in vielen Bereichen. Die Autoren (Gao et al., 2024; Lewis et al., 2021; Salemi & Zamani, 2024) kommen zu dem Schluss, dass durch RAG die LLMs bessere Ergebnisse liefern können. Ergebnisse, welche RAG benutzen, zeichnen sich bei ihrer Performance aus und sind spezifischer sowie faktenreicher als Ergebnisse, die nur von dem LLM ohne RAG erzeugt wurden. (Gao et al., 2024; Lewis et al., 2021; Salemi und Zamani, 2024)

2.4.1 Einsatzmöglichkeiten von RAG

RAG spielt in dieser Arbeit eine entscheidende Rolle. Mithilfe von RAG kann dem LLM eine Anforderungsspezifikation zur Verfügung gestellt werden, aus welcher bereits manuell Anforderungstestfälle generiert wurden. Durch RAG kann dem LLM diese Anforderungsspezifikation, welche eine PDF-Datei ist, verarbeiten und daraus Informationen ziehen. (Gao et al., 2024; Wu et al., 2024)

In Kapitel 4.4.2 wird gezeigt, wie Testfälle auf Basis ihrer Korrektheit ausgewertet werden. RAG kann eine Rolle spielen, die Korrektheit von KI-Antworten zu verbessern und somit die durch GPT-4o von OpenAI (o. D.) generierten Testfälle zu verbessern. Wie in Kapitel 4.1 steht, ist die Fähigkeit eines LLM die Methode RAG zu benutzen, ist eine Bedingung um ein ein passendes LLM zu finden.

2.5 Testfall

Es ist entscheidend zu definieren, was ein Testfall auszeichnet und welche Grundlagen und Definitionen für dessen Aufbau gegeben sind. Ohne dieses Wissen ist es unmöglich, eine Metrik für die Auswertung und Qualität der generierten Testfälle zu definieren.

In dieser Arbeit werden Testfälle mithilfe dem LLM GPT-4o erzeugt. Ein Testfall, ist wie bereits der Name sagt, ein Szenario, in welchem getestet wird, ob die Anwendung die Funktionalitäten so erfüllt, wie das von ihr erwartet wird. Für Testfälle gibt es eine internationale und standardisierte Definition und ein Regelwerk von ISO/IEC/IEEE 29119-1:2022-01 (2022). Eine Übersicht, welche die Testbegrifflichkeiten genau erläutert, findet sich in dem ISTQB/GTB Standardglossar der Testbegriffe (2017). Dieses Standardglossar beschreibt dabei sämtliche Begrifflichkeiten auf Englisch und ihrer offiziellen Übersetzung auf Deutsch.

2.5.1 Aufbau eines Testfalls

Ein Testfall besitzt eine ID, einen Namen, Vorbedingungen, welche erfüllt sein müssen, damit der Testfall durchgeführt werden kann, ein Szenario als Beschreibung, Testschritte die nummeriert und aufeinander aufbauen und erwartete Ergebnisse, wie die Anwendung handeln wird, wenn diese Schritte ausgeführt werden. ISO/IEC/IEEE 29119-1:2022-01 (2022). Folgende Grafik zeigt ein Testfalldesign für eine SAP-Applikation:

Test Case Titel		Upload external catalog to Ariba Network		
Preconditions		Ariba network users with the required permissions Catalog file in the desired format		
Testobject		Ariba Catalog		
Goals for this Test Case		1. Functional goal: Successfully upload of a catalog file to Ariba Network. 2. Nonfunctional goal: The file can be uploaded within 10 seconds.		

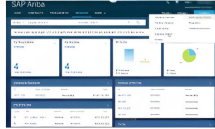
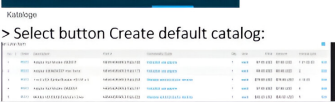
Step	Description (Perform actions and screenshots)	Executable (Transaction, Fiori, URL, etc.)	Possible test data	Expected result (Is required to detect defects)
01	Log in Log in to the system as an external supplier with <username> and <password>.	http://supplier.ariba.com		You will see "cockpit" home site. 
02	Go to Upload external catalog view Select menu <catalogs>  > Select button Create default catalog:			View «details» appears: 
03	Create a new catalog Enter <catalog name> Formulate a <description> Click <save>			Catalog has been successfully uploaded and saved. The status of the catalog is "published".

Abbildung 2.7: Die Grafik von Enderli (2019) zeigt einen Testfall im SAP Solution Manager

In den Testschritten wird überprüft, ob die Anwendung die Anforderungen erfüllt und somit das erwartete Ergebnis eintrifft. Testfälle können dabei sowohl positiv als auch negativ sein. Bei negativen Testfällen wird überprüft, ob das erwartete Ergebnis nicht eintrifft, da die Voraussetzungen nicht erfüllt sind. In Kapitel 4.4.1 steht, dass ein Testfall der von GPT-4o von OpenAI (o. D.) generiert wird, die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) besitzen muss. Dies ist wichtig für die Dokumentation der Testfälle. (ISO/IEC/IEEE 29119-1:2022-01, 2022; ISTQB/GTB Standardglossar der Testbegriffe, 2017)

2.6 Anforderungsspezifikation

Eine Anforderungsspezifikation ist das Fundament, aus dem Testfälle erstellt werden. Ein Testfall überprüft, ob die Anforderungen aus dieser Spezifikation erfüllt werden. Demnach ist eine Anforderungsspezifikation ein Dokument, welches alle Anforderungen enthält, welche das System erfüllen muss und dessen Erfüllbarkeit und nicht Erfüllbarkeit mit Testfällen abgedeckt und getestet werden. Da sie der Grund für die Testfälle sind, müssen Anforderungsspezifikationen vollständig, präzise und in sich konsistent sein. Anforderungsspezifikationen enthalten System- und Benutzeranforderungen. Systemanforderungen beschreiben dabei die Benutzeranforderungen genau, wobei die Benutzeranforderungen funktionale und nicht funktionale Anforderungen besitzen. Folgende Grafik veranschaulicht anhand eines Beispiels den Unterschied zwischen funktionalen und nicht funktionalen Anforderungen:

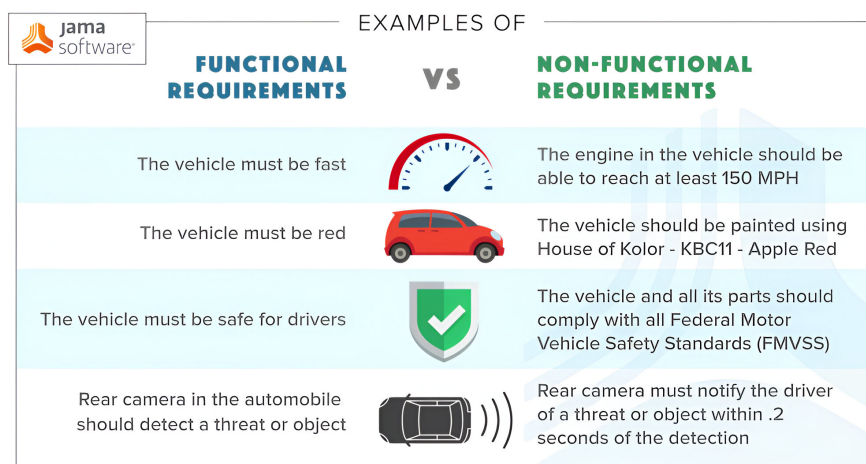


Abbildung 2.8: Die Grafik von Jama Software (o. D.) zeigt die Eigenschaften von funktionalen vs. nicht funktionalen Anforderungen

Nicht funktionale Anforderungen beschreiben laut Visure (o. D.) zum Beispiel die Bereiche Sicherheit, Wartung und Kriterien an das System. In dieser Arbeit geht es um funktionale Anforderungen. Diese legen fest, wie das System funktioniert und was passiert, wenn etwas im Positiven als auch wenn etwas im Negativen geschieht. Der Unterschied ist an der Grafik von Jama Software (o. D.) zu erkennen. Während die funktionellen Anforderungen festlegen, dass das Auto schnell sein und eine rote Farbe besitzen muss, legen die nicht funktionalen Anforderungen exakt fest, welche Geschwindigkeit das Auto erreichen und welchen genauen Rotton dieses besitzen muss. (Barmi et al., 2011; Mustafa et al., 2021)

2.6.1 Bedeutung der Anforderungsspezifikation für die Testfallgenerierung

Anforderungsspezifikationen dienen als Grundlage für die Testfallgenerierung durch GPT-4o von OpenAI (o. D.) in dieser Arbeit. Sie werden GPT-4o per RAG übermittelt und per Prompt dazu aufgerufen, Testfälle aus diesen zu generieren, welche alle Bereiche pro Testszenario abdecken. Dabei werden sie mit Screenshots der getesteten Webseite ergänzt. Anforderungsspezifikationen sind essenziell für diese Arbeit. Sie dienen als Grundlage zur Testfallgenerierung und die von GPT-4o generierten Testfälle werden mithilfe von manuell erstellten Testfällen auf ihre Richtigkeit überprüft. Diese dienen als Benchmark und wurden aus den gleichen Anforderungsspezifikationen erstellt, welche GPT-4o zur Verfügung gestellt werden.

3 Related Work und aktuelle Forschung

In dieser Arbeit wird das LLM GPT-4o von OpenAI (o. D.) verwendet, um Anforderungstestfälle zu generieren. Diese Arbeit benutzt somit ein Sprachmodell zur Textgenerierung, da Testfälle in Form eines Textes erstellt werden. Für das Vorgehen bei der Erstellung von Testfällen und für das Aufstellen von Kriterien für die Auswertung der Testfälle in Kapitel 4.4 ist es wichtig, wissenschaftliche Arbeiten im Bereich der Textgenerierung und Testfallgenerierung mithilfe von LLMs zu berücksichtigen. In diesem Kapitel werden Arbeiten, welche sich mit diesen Bereichen beschäftigen, erläutert und deren Ergebnisse eingeordnet.

Für den Einstieg zu dem Thema eignet sich ein wissenschaftliches Paper von Iqbal und Qureshi (2022), welches einen Überblick im Bereich Textgenerierung bietet. Dieses Paper stellt viele Deep Learning Modelle vor, die für die Generierung von Text verwendet werden. Dabei werden die verschiedenen Modelle zusammengefasst und das Paper ermöglicht einen detaillierten Überblick über die Vergangenheit, Gegenwart und Zukunft von Textgenerierungsmodellen im Deep Learning. Die Autoren Iqbal und Qureshi (2022) gehen davon aus, dass in Zukunft immer bessere Modelle entwickelt werden und dass die Forschung im Bereich der Texterzeugung weiter vorangetrieben wird. Das Paper macht jedoch auch deutlich, dass Modelle bei längeren Texten die natürliche Sprache noch nicht vollständig begriffen haben. (Iqbal und Qureshi, 2022)

Es ist wichtig, diese Erkenntnis zu berücksichtigen, wenn es darum geht, die Prompts zu schreiben und der KI die Anforderungsspezifikationen zu übermitteln. Auch die Anzahl der von GPT-4o auf einmal generierten Testfälle kann eine Auswirkung auf die Qualität der Testfälle haben.

Nicht nur die Textgenerierung durch KI ist wichtig, sondern auch die Auswertung des generierten Textes. Yuan et al. (2021) stellten sich die Frage, wie die generierten Texte nach Flüssigkeit, Genauigkeit und Effektivität beurteilt werden können. Dafür entwickelten sie eine Metrik: Der sogenannte „BARTSCORE“ bewertet flexibel und unüberwacht aus unterschiedlichen Blickwinkeln. Unter anderem betrachtet er Informativität, Geläufigkeit oder Faktizität der generierten Texte. Yuan et al. (2021) kommen zu dem Ergebnis, dass ihr Modell in 16 von 22 Settings besser abgeschnitten hat als andere bewertete Metriken. (Yuan et al., 2021)

Diese Erkenntnis zeigt, wie wichtig es für die Auswertung der Arbeit ist, eine Metrik zu verwenden, welche die Qualität des generierten Textes und damit die Qualität der generierten Testfälle auswertet.

Die Werke von (Lazić, 2013; Nirpals & Kale, 2011) liefern einen Überblick über vorhandene Metriken im Bereich des Testens. Die Autoren der beiden Werke (Lazić, 2013; Nirpals & Kale, 2011) kommen zu dem Schluss, dass Metriken wichtig für das Testen sind. Die in beiden Werken vorkommende Metrik zur Messung des Automation Coverage dient in Kapitel 4.4.3 als Grundlage, um das Qualitätskriterium Abdeckung zu messen. (Nirpals und Kale, 2011; Lazić, 2013)

Diese Werke zeigen, dass es im Bereich des Testens viele Metriken und Kriterien gibt, anhand derer die Qualität von Testfällen ausgewertet werden können. Es ist somit wichtig, klar zu beschreiben mit welchen Metriken, die Qualitätskriterien in dieser Arbeit ausgewertet werden.

Das Werk von Tran et al. (2025) untersucht die wichtigsten Qualitätskriterien aus der Anwendersicht, da es sehr viele Qualitätskriterien gibt. Die Autoren Tran et al. (2025) kommen dabei zu dem Schluss, dass die Qualitätskriterien Coverage (Deutsch: Abdeckung), Fault Detection (Deutsch: Fehlererkennung), Maintainability (Deutsch: Wartbarkeit), Reliability (Deutsch: Zuverlässigkeit) und Usability (Deutsch: Benutzerfreundlichkeit), die wichtigsten Kriterien für Testfälle sind. Der Stellenwert der Qualitätskriterien ist dabei jedoch abhängig vom Kontext der Tests. (Tran et al., 2025)

Die Autoren Li et al. (2024) erforschen das Thema Halluzinationen im Bereich der LLM. Das Ziel ist es, Halluzinationen zu erkennen und die Anzahl der Halluzinationen zu reduzieren. Dabei kommen Li et al. (2024) zu dem Schluss, dass die Anzahl der Halluzinationen durch gezieltes Training und RAG verringert werden kann. Da in dieser Arbeit das LLM GPT-4o verwendet wird, ist es wichtig, das Phänomen der Halluzination zu beachten. (Li et al., 2024)

Im Bereich des Testens gibt es unterschiedliche Bereiche. Für diese Arbeit ist es relevant herauszufinden, ob bereits KI oder LLMs im Bereich des Testens und zur Testfallerstellung eingesetzt werden. Folgende Arbeiten zeigen den Einsatz von KI oder LLMs im Bereich des Testens.

Das Ziel der Arbeit von Khan et al. (2024) ist es, einen Überblick über die Rolle von KI bei der Automatisierung von Softwaretests zu geben. Dabei stellen die Autoren Khan et al. (2024) die These auf, dass das Ziel der automatisierten Softwaretesterstellung mit KI möglich sein kann. Im Werk werden zum Schluss entscheidende Schwierigkeiten, Probleme und Voraussetzungen vorgestellt. Am Ende bekräftigen Khan et al. (2024) jedoch ihre These, dass das Testen von Software mit einer Reihe von Automatisierungsgadgets möglich sein kann. (Khan et al., 2024)

Die Autoren Bhandari et al. (2024) beschäftigen sich mit dem Einsatz und dem Potenzial von LLMs im Bereich des Chip-Testing. Dabei sind funktionale Tests, die sich auf Testbenches stützen, wichtig für das Chipdesign. Es wird Feedback in das LLM eingegeben, um so die Testbench-Generierung zu verbessern. Dies findet iterativ statt, um die Testabdeckung zu verbessern. Bhandari et al. (2024) nennen als Herausforderungen die Wiederholung der Antworten und das Finetuning des LLM bei dem Umfang ihrer Arbeit. Laut Bhandari et al. (2024) gestaltet sich die Wertung der Testbenches in Bezug auf die Abdeckung nicht einfach. Trotzdem kommen Bhandari et al. (2024) zum Schluss, dass durch das Feedback die Testbenches besser verstanden werden. Ein entscheidender Faktor für die Qualität der Ausgabe ist für Bhandari et al. (2024) dabei der Prompt. (Bhandari et al., 2024)

In der Arbeit von Ouédraogo et al. (2024) wird die Wirksamkeit von LLM bei der Erstellung von Unit Tests untersucht. Unit Tests sind ein Framework zur Entwicklung von Softwaretests. Dabei kommen Ouédraogo et al. (2024) zu dem Schluss, dass sich die mit LLM erzeugten JUnit Tests mit den von Menschen geschriebenen Tests in Bezug auf Codierungsstandards ähneln. (Ouédraogo et al., 2024)

Die Autoren S. Gu et al. (2024), welche sich in ihrer Arbeit ebenfalls mit dem Einsatz von KI bei der Erstellung von Unit Tests beschäftigen, nennen drei mögliche Probleme bei der Erzeugung von Testfällen. Ein unzureichender Kontext sowie fehlende Test- und Abdeckungsinformationen. Als Lösung bieten S. Gu et al. (2024) das Tool TestART an, welches eine Erfolgsquote von 78,55 % aufweist. Laut S. Gu et al. (2024) erzielt TestART damit eine 18% höhere Erfolgsquote und eine um 20% höhere Abdeckungsrate als herkömmliche Modelle. (S. Gu et al., 2024)

Mit dem Thema des automatischen Benutzeroberflächen (GUI)-Testings beschäftigt sich die Arbeit von Z. Liu et al. (2024). Dabei sehen Z. Liu et al. (2024) einen großen Fortschritt im Bereich des automatischen GUI-Testing. Die Autoren Z. Liu et al. (2024) sind zu der Erkenntnis gekommen, dass von LLM erzeugte Tests noch eine geringe Aktivitätsabdeckung aufweisen. Dies bedeutet, dass nur ein geringer Teil der möglichen Benutzeraktionen und Interaktionen mit der GUI abgedeckt ist. Dadurch kann mit diesen Tests nicht gewährleistet werden, dass alle Funktionen ordnungsgemäß und nach Vorgabe funktionieren. Z. Liu et al. (2024) schlagen deshalb GPTDroid vor, mit welchem sie in ihrer Arbeit eine 75 % Aktivitätsabdeckung in 93 Apps erzielen. (Z. Liu et al., 2024)

Die Arbeit von Kirinuki und Tanno (2024) vergleicht durch GPT-4 generierte Testfälle mit manuell erstellten Testfällen. Dabei handelt es sich um Black-Box-Tests. Die Autoren Kirinuki und Tanno (2024) kommen zu dem Ergebnis, dass beim Aspekt der Testabdeckung die von GPT-4 erzeugten Testfälle gleich gut oder sogar besser abschneiden als von Menschen erstellte Testfälle. Diese Testabdeckung wird laut Kirinuki und Tanno (2024) noch erhöht, wenn ein Mensch zusammen mit GPT-4 Testfälle erstellt. (Kirinuki und Tanno, 2024)

Diese Werke zeigen, dass KI und LLMs bereits in verschiedenen Testbereichen eingesetzt werden. Dabei handelt es sich um die Bereiche der Softwaretests, des Chip-Testing, der Unit Tests, des GUI-Testing und der Black-Box-Tests. Dies zeigt, dass es möglich ist, dass LLM wie GPT-4o in der Lage ist, Anwendungstestfälle zu generieren, welche eine bessere Qualität besitzen als manuell erstellte Anwendungstestfälle.

4 Methodik

Kapitel 4 zeigt die Methodik der Arbeit, es beschreibt das verwendete LLM GPT-4o von OpenAI (o. D.). Anschließend werden die Testdaten erläutert. Dabei gibt es Anforderungsspezifikationen, welche GPT-4o zu Verfügung gestellt werden, um Testfälle zu generieren. Manuelle Testfälle dienen zusammen mit der getesteten Anwendung als Benchmark, um diese auf ihre Korrektheit, Logik und Abdeckung zu vergleichen. Dabei wurde nach Testdaten gesucht, die sowohl selbst Open Source sind und auch Open Source Projekte testen, damit sie frei verfügbar sind. Sie müssen Anforderungsspezifikationen und manuelle Testfälle enthalten. Dabei müssen die Testfälle die Testfallstruktur von ISO/IEC/IEEE 29119-1:2022-01 (2022) aufweisen. Außerdem ist es wichtig, dass sie verschiedene Testszenarien abdecken, um für mehrere Anforderungen als Benchmark genutzt werden zu können. Die Prompts werden erklärt und eine Metrik aufgestellt, nach der die generierten Testfälle ausgewertet und bewertet werden.

4.1 Verwendetes LLM

In Kapitel 2 werden Methoden zur Testfallgenerierung genannt. Ein geeignetes LLM für die Erstellung der Testfälle in dieser Arbeit soll diese Bedingungen erfüllen:

- Das Modell muss frei verfügbar sein oder geringe Kosten aufweisen, damit Nutzer die Erkenntnisse aus dieser Arbeit nutzen und weiterverwenden können.
- Das LLM muss Retrieval-Augmented Generation (RAG) verwenden und in der Lage sein Bilder zu erkennen, um effiziente Testfälle aus den Anforderungsspezifikationen zu generieren.

Für diese Arbeit wird GPT-4o von OpenAI (o. D.) verwendet, da es diese Punkte erfüllt. GPT-4o ist kostenlos und öffentlich nutzbar. Die Nutzung ist jedoch für den kostenlosen Zugang nur eingeschränkt. Die uneingeschränkte Version kostete am 08.02.2025 20 US-Dollar im Monat. Wie Backlinko (2025) berichtet, hatte ChatGPT von OpenAI (o. D.) im Januar 2025 nach eigenen Angaben über 300 Millionen Nutzer. Somit ist GPT-4o ein bekanntes LLM. Nutzer können es mit einer grafischen Benutzeroberfläche nutzen und es als Werkzeug zur Testfallgenerierung benutzen.

Da GPT-4o Bilder erkennen und mit Dokumenten arbeiten kann, ist es in der Lage, die in Kapitel 2.4 erklärte Methode RAG zu verwenden. Es kann somit Informationen aus den in Kapitel 4.2 beschriebenen Anforderungsspezifikationen ziehen und dabei Webseiten und ihr Verhalten in Form von Screenshots erkennen und damit arbeiten. Außerdem besitzt GPT-4o eine Erinnerungsfunktion, mit welcher es auf wichtige Anmerkungen und Informationen aus vorherigen Prompts zurückgreifen kann.

Es muss nicht mit viel Aufwand, Zeit und Ressourcen antrainiert oder ein eigenes Modell entwickelt werden. GPT-4o ist in vielen Sprachen nutzbar und kann so von Nutzern weltweit verwendet werden.

In Kapitel 3 wird aufgezeigt, dass die Autoren Kirinuki und Tanno (2024) GPT-4o benutzen um Black-Box-Tests zu erzeugen und dass diese im Bereich der Testabdeckung, eine gleich gute oder sogar bessere Abdeckungsrate erreichen, als von Menschen erstellte Testfälle. Diese Erkenntnisse zeigen, dass GPT-4o sich als LLM für diese Arbeit eignet, da es die zwei Bedingungen erfüllt und bereits in einem ähnlichen Bereich von Kirinuki und Tanno (2024) eingesetzt wurde.

Mit einem Prompt kann GPT-4o nun Testfälle erzeugen. Wie die Testdaten aufgebaut sind, welcher der Prompt benutzt wird und nach welchen Kriterien die Ergebnisse eingeordnet und ausgewertet werden, wird in Kapitel 4.2, Kapitel 4.3 und Kapitel 4.4 beschrieben.

4.2 Testdaten

Als Grundlage zur Erstellung von Testfällen dienen Anforderungsspezifikationen. Um die Korrektheit, Logik und Abdeckung der von GPT-4o von OpenAI (o. D.) generierten Testfälle messen zu können, ist es essenziell, dass manuelle Testfälle verfügbar sind, welche bereits aus den gleichen Anforderungsspezifikationen erstellt wurden, aus denen GPT-4o Testfälle erzeugt hat. Dadurch können die von GPT-4o generierten Testfälle in Bezug auf ihre Abdeckung, Factual Correctness und Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) bewertet werden. Für diese Arbeit wird für die Gewinnung von Testdaten, das Open Source Projekt von Pavankumar21github (2022) auf GitHub verwendet, welches bereits Anforderungsspezifikationen und manuelle Testfälle beinhaltet.

Die Testdaten und die Website sind somit frei verfügbar und können in dieser Arbeit verwendet werden. Die Open Source Onlineshop Plattform OpenCart (o. D.) besitzt viele Funktionen und Anwendungen, welche Pavankumar21github (2022) in 31 verschiedenen Testszenarien mit insgesamt 507 Testfällen testet. Damit eignen sich das Projekt und die Webseite zur Erstellung von Anwendungstestfällen und als Benchmark, um diese auszuwerten. Spezifikationen wie diese werden GPT-4o zur Verfügung gestellt. Die generierten Testfälle können nun mit den manuellen Testfällen, die die gleichen Funktionen und Anforderungen abdecken, auf ihre Korrektheit, Logik und Abdeckung verglichen werden.

Folgende Grafik zeigt einen Testfall, welcher von Pavankumar21github (2022) erstellt wurde und welcher die Erstellung eines Accounts in OpenCart (o. D.) testet.

Test Case Title		Pre-requisites	Test Steps			
Validate Registering an Account by providing only the Mandatory fields		1. Open the Application (https://demo.opencart.com) in any Browser	1. Click on 'My Account' Drop menu 2. Click on 'Register' option 3. Enter new Account Details into the Mandatory Fields (First Name, Last Name, E-Mail, Telephone, Password, Password Confirm and Privacy Policy Fields) 4. Click on 'Continue' button (ER-1) 5. Click on 'Continue' button that is displayed in the 'Account Success' page (ER-2)			
Test Data	Expected Result (ER)		Actual Result	Priority	Result	Comments
Not Applicable	1. User should be logged in, taken to 'Account Success' page and proper details should be displayed on the page 2. User should be taken to 'Account' page and a confirm email should be sent to the registered email address		1. User got logged in, taken to 'Account Success' page successfully and proper details got displayed on the page 2. User is taken to 'Account' page and a confirm email received at the registered email address	P1	PASS	

Abbildung 4.1: Ein von Pavankumar21github (2022) erstellter Testfall, welcher die Erstellung eines Accounts in OpenCart (o. D.) anhand der oben gezeigten Anforderungsspezifikation testet und als Benchmark für die von GPT-4o erstellten Testfälle dient

Wie in der Grafik zu erkennen ist, verwendet Pavankumar21github (2022) die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022). Diese Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) ist wie in Kapitel 4.4.1 beschrieben, eine Voraussetzung, welche die von GPT-4o erstellten Testfälle erfüllen müssen. Dabei sind die Testschritte von Pavankumar21github (2022) in sich logisch, bauen aufeinander auf und stimmen mit den Informationen auf der OpenCart (o. D.) Webseite überein. Sie dienen deshalb zusammen mit den Informationen aus der OpenCart (o. D.) Webseite als Benchmark für die von GPT-4o erstellten Testfälle. Das Projekt erfüllt somit die genannten Voraussetzungen und liefert gleichzeitig eine Zahl an Testdaten, um sie mit den durch GPT-4o generierten Testfälle zu vergleichen. Obwohl in dieser Arbeit nur ein Projekt als Referenz verwendet wird, bietet es mit 31 unterschiedlichen Testszenarien genügend Material, um zu fundierten Ergebnissen zu kommen. Mithilfe dieser Anforderungsspezifikationen und mit für das Testszenario relevante Screenshots, generiert GPT-4o Testfälle. Wie dieser Vorgang genau im Detail vonstatten geht, wird in Kapitel 4.3 erläutert. Diese generierten Testfälle werden in Kapitel 4.4 mithilfe der manuellen Testfälle auf ihre Korrektheit, Logik und Abdeckung überprüft und gemessen. Die manuellen Testfälle von Pavankumar21github (2022) dienen dabei als Benchmark.

4.3 Prompt und verwendete Methoden

In diesem Kapitel wird beschrieben, wie in dieser Arbeit Testfälle durch GPT-4o von OpenAI (o. D.) generiert werden. Zuerst wird die Erinnerungsfunktion aktiviert, damit GPT-4o auf Informationen zurückgreifen kann, welche GPT-4o bereits in anderen Prompts mitgegeben wurden. Anschließend werden GPT-4o Screenshots von dem jeweiligen Testszenario von der OpenCart (o. D.) Webseite übergeben. Dadurch erhält es die Informationen darüber, wie die Seite funktioniert und reagiert und kann somit Anwendungen testen. Danach wird GPT-4o die jeweilige Anforderungsspezifikation als PDF-Datei übergeben. Durch die in Kapitel 2.4 beschriebene Methode RAG kann es dadurch die Informationen besser bearbeiten. Anschließend wird ein Prompt übergeben, welcher in folgender Grafik gezeigt wird.

Ich habe dir eine Anforderungsspezifikation abgelegt, welche Informationen über Register Account enthält. Sowie Screenshots, welche die Funktionen und die Reaktionen der OpenCart-Webseite zeigen. Schreibe daraus Testfälle, die alle Szenarien abdecken, welche getestet werden können.

Die Testfälle sind aus der Anwendersicht zu schreiben und müssen die Testfallstruktur aus den ISO/IEC/IEEE Richtlinien erfüllen.

Benutze nur Informationen, welche in der Anforderungsspezifikation oder den Screenshots zu finden sind.

Gehe dabei Step by Step vor und schreibe die Testfälle in Tabellenform und auf Englisch.

Abbildung 4.2: Ein Prompt an GPT-4o

GPT-4o wird dazu aufgefordert alle Funktionen pro Testszenario als jeweils einzelnen Testfall zu generieren. Es soll dabei nur Informationen aus der Anforderungsspezifikation und aus den Screenshots benutzen und dabei die Struktur aus ISO/IEC/IEEE 29119-1:2022-01 (2022) erfüllen. Um auf mögliche Fehler und Halluzinationen vorzubeugen, wird GPT-4o dazu aufgefordert, die Testfälle in Tabellenform anzuzeigen. Es kann so auf mögliche Fehler oder fehlende Testfälle hingewiesen werden die GPT-4o beheben soll. Die Testfälle werden dann in einer Excel-Tabelle dokumentiert. Anschließend werden diese ausgewertet. Die Metriken zu der Auswertung der generierten Testfälle finden sich in Kapitel 4.4. Dabei dienen die manuellen Testfälle von Pavankumar21github (2022) und Informationen auf der OpenCart (o. D.) Webseite als Referenz.

4.4 Qualitätskriterien

Im Bereich des Testens gibt es viele Metriken und Qualitätskriterien, anhand derer gemessen werden kann, wie ein Testfall abschneidet. Anhand dieser hier dargestellten Qualitätskriterien werden die von GPT-4o erstellten Testfälle ausgewertet und nach ihrer Qualität in Bezug auf Korrektheit, Logik und Abdeckung gemessen. Im Folgenden werden die Qualitätskriterien aufgestellt, um die durch GPT-4o erstellten Testfälle auszuwerten.

4.4.1 Testfallstruktur

In Kapitel 2.5 wird die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) beschrieben. Ein Testfall nach ISO/IEC/IEEE 29119-1:2022-01 (2022) besitzt folgende Merkmale: Eine ID, einen Namen, Vorbedingungen, ein Szenario als Beschreibung, Testschritte die nummeriert sind und ein erwartetes Ergebnis. In dieser Arbeit wird von GPT-4o erwartet, dass es, wenn es einen Testfall generiert, diesen Aufbau einhält. Der Grund ist folgender: Für viele Nutzer ist es wichtig, dass Standards eingehalten werden. Ein Testfall, welcher vom Aufbau her diesen Standard nicht erfüllt, ist nicht brauchbar und muss somit auch negativ bewertet werden.

Folgendes Beispiel zeigt einen Testfall, welcher die Anforderungen aus der in Kapitel 4.2 gezeigten Anforderungsspezifikation testen soll, welcher nicht die Testfallstruktur erfüllt und somit nicht weiter berücksichtigt wird. Dies liegt daran, dass dieser keine ID und kein Name besitzt und es somit nicht möglich ist diesen strukturiert zu dokumentieren.

Testfall	Ein Benutzer erstellt ein Konto während des Checkouts.
Schritte	1. Gehe zur Seite. 2. Wähle Register. 3. Weiter drücken. 4. Daten eingeben. 5. Weiter.
Erwartetes Ergebnis	Sollte funktionieren.

Abbildung 4.3: Ein von GPT-4o erstellter Testfall, welcher nicht der Testfallstruktur entspricht

Bei der Auswertung wird überprüft und gemessen, ob GPT-4o standardmäßig die Testfallstruktur von ISO/IEC/IEEE 29119-1:2022-01 (2022) benutzt. Das Qualitätskriterium Testfallstruktur ist entscheidend für diese Arbeit und die Auswertung der Testfälle. Erst wenn die generierten Testfälle diesen Aufbau besitzen, können diese für weitere Qualitätskriterien ausgewertet werden. Deshalb wird überprüft und gemessen, wie viele der von GPT-4o generierten Testfälle diese Testfallstruktur ganz oder teilweise besitzen.

Der Wert Testfallstruktur hat zwei Kategorien, welche in folgender Tabelle abgebildet sind:

Testfallstruktur:

	1. Positiv	2. Negativ
Einstufung	0 Merkmale fehlen	$1 \geq$ Merkmale fehlen

Die Testfälle werden pro Testszenario ausgewertet. Am Ende wird angegeben, wie viel Prozent der Testfälle in der positiven oder in der negativen Kategorie pro Testszenario eingeordnet werden. Dies erfolgt folgendermaßen: Die Anzahl der in der ersten Kategorie eingeordneten Testfälle pro Testszenario wird mit der Gesamtzahl der Testfälle pro Testszenario dividiert. Zum Beispiel sind, wenn es in einem Testszenario acht positiv und zwei negativ eingestufte Testfälle gibt, die Testfälle in diesem Testszenario zu 80 % positiv. Diese Berechnung wird pro Testszenario durchgeführt und am Ende wird der Mittelwert aller Testszenarien bestimmt.

4.4.2 Factual Correctness

Die Inhalte eines Testfalls müssen stimmig sein und den Prozess in der richtigen Reihenfolge wiedergeben. Einfaches Beispiel: Ein Nutzer kann sich nicht von einer Anwendung abmelden, wenn er sich nicht vorher angemeldet hat. Ein Testfall muss somit als ersten Testschritt die Anmeldung besitzen, bevor dieser den Testschritt die Abmeldung hat. Deshalb ist ein Testfall unbrauchbar, wenn dieser nicht die Logik der Anforderungsspezifikation erfüllt. Um die Logik und die Korrektheit der generierten Testfälle bestimmen zu können, werden die manuell erstellten Testfälle von Pavankumar21github (2022), zusammen mit den Informationen auf der OpenCart (o. D.) Webseite als Benchmark genutzt. Diese werden pro Testszenario mit den generierten Testfällen auf ihre Korrektheit überprüft.

Ein weiteres Problem für die Factual Correctness ist das Problem der Halluzination. In Kapitel 1.1 steht, dass Siebert (2024) den Begriff Halluzination als Phänomen beschreibt, bei dem eine KI-Antwort nicht mit den gegebenen Informationen übereinstimmt und die KI sich Antworten generiert, die faktisch nicht richtig sind.

Mit dem Qualitätskriterium Factual Correctness von Ragas (2024) wird in dieser Arbeit der Wert der faktisch korrekten Testschritte pro Testfall und der Gesamtzahl an Testschritten des Testfalls gemessen. Ein faktisch korrekter Testschritt steht in der richtigen Reihenfolge, ist keine Halluzination und besitzt seine Informationen nur aus der Anforderungsspezifikation oder dem Screenshot aus der Webseite.

Nach Ragas (2024) misst sich die Factual Correctness folgendermaßen:

- **Precision** (P): Der Anteil der generierten Testfälle, die als korrekt eingestuft wurden und tatsächlich korrekt sind.

$$P = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall** (R): Der Anteil der tatsächlich korrekten Testfälle, die von der KI erkannt wurden.

$$R = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **F1** ($F1$): Mit dem F1 Wert lässt sich die Factual Correctness bestimmen. Die Werte liegen zwischen 0 und 1 und je näher der Wert an 1 geht, desto mehr Factual Correctness besitzt dieser Testschritt.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Dabei sind in dieser Arbeit an Ragas (2024) angelehnt:

- True Positive (TP) Anzahl faktisch korrekter Testschritte pro Testfall,
- False Positive (FP) Anzahl faktisch nicht korrekter Testschritte pro Testfall, d.h. Testschritte welche sich nicht in der Anforderungsspezifikation oder den Screenshots des Testszenario befinden,
- False Negative (FN) die Anzahl der Testschritte, welche sich entweder in der Anforderungsspezifikation oder den Screenshots des Testszenario, jedoch nicht in dem von GPT-4o generierten Testfall befinden.

Durch diese Metrik kann der Anteil der faktisch korrekten Testschritte pro Testfall in Prozent gemessen werden. Um die Factual Correctness der Testfälle zu bestimmen, werden die von GPT-4o generierten Testfälle auf OpenCart (o. D.) überprüft, ob die Informationen stimmen und alle Informationen pro Testschritt so auf OpenCart (o. D.) zu finden sind. So kann überprüft werden, ob GPT-4o Testschritte außer Acht lässt oder in einem Testschritt falsche Begriffe oder Schritte vorkommen. Auch ob es Funktionen testet, die weder in der Anforderungsspezifikation oder auf der Website zu finden sind. Ein Testschritt, welcher in der falschen logischen Reihenfolge steht, ist ebenfalls nicht faktisch korrekt. Weist ein Testfall ein falsches oder generisches „Erwartetes Ergebnis“ auf, ist er auch negativ. Die Testfälle werden folgendermaßen eingeordnet:

Factual Correctness:

	1. Positiv	2. Negativ
F1-Wert	1	1 <
Erwartetes Ergebnis	Korrekt	Falsch oder generisch

Es wird eingeordnet, wie viel Prozent der Testfälle, in welcher der zwei Kategorien eingeordnet werden. Ein Testfall muss beide Bedingungen erfüllen um positiv zu sein.

Anschließend wird der Mittelwert berechnet, um die Gesamtrate aller Testfälle zu ermitteln. Von einem Testfall wird erwartet, dass er sich in der positiven Kategorie befindet.

Folgendes Beispiel zeigt einen von GPT-4o erstellten Testfall, welcher sowohl die richtige Testfallstruktur als auch nur faktisch korrekte Testschritte aufweist.

Testfall-ID	TC-001
Testfall-Name	Registrierung eines neuen Kunden im Checkout-Prozess
Beschreibung	Dieser Testfall überprüft, ob ein neuer Kunde erfolgreich ein Konto erstellen und den Checkout-Prozess fortsetzen kann.
Voraussetzungen	- Der Benutzer hat Zugriff auf die Checkout-Seite. - Der Benutzer hat noch kein Konto.
Testschritte	1. Öffne die Checkout-Seite. 2. Wähle die Option „Register Account“. 3. Klicke auf „Continue“. 4. Fülle alle erforderlichen Felder im Abschnitt „Account & Billing Details“ aus (Name, Adresse, E-Mail, Passwort). 5. Klicke auf „Continue“. 6. Überprüfe, ob der Benutzer erfolgreich zum nächsten Schritt des Checkouts weitergeleitet wird.
Erwartetes Ergebnis	Der Benutzer wird erfolgreich registriert und kann mit dem Checkout-Prozess fortfahren.

Abbildung 4.4: Ein von GPT-4o erstellter Testfall, welcher der Testfallstruktur entspricht und nur faktisch korrekte Testschritte besitzt

Nur wenn ein Testfall die richtige Teststruktur und faktisch korrekte Testschritte besitzt, ist es sinnvoll, ihn auf seine Qualität in Bezug auf die Abdeckung auszuwerten. Es wird gemessen, wie viele Testfälle pro Testszenario positiv sind. Dabei wird am Ende ein Mittelwert über alle Testszenarien hinweg ermittelt. Um die Streuung zu bestimmen, wird ein Säulendiagramm erstellt, um die Werte, je nach Testszenario, darzustellen.

4.4.3 Coverage (Deutsch: Abdeckung)

Um das Kriterium Coverage zu berechnen, wird die Metrik für die Automation Coverage nach (Lazić, 2013; Nirpals & Kale, 2011) verwendet. In dieser Arbeit wird jedoch nicht die Automation Coverage berechnet, sondern die Anzahl der von GPT-4o erstellten Testfälle mit der Anzahl der manuellen Testfälle pro Testszenario berechnet.

Die Abdeckung wird für jedes Testszenario einzeln bewertet und danach wird der Mittelwert aus allen Testfällen berechnet. Die Streuung pro Testszenario wird mit einem Säulendiagramm dargestellt.

Für die Abdeckung wird die Formel nach (Lazić, 2013; Nirpals & Kale, 2011) verwendet, welche für den Anwendungsfall in dieser Arbeit angepasst wurde, da nicht die Abdeckung der Automation, sondern die Abdeckung der von GPT-4o erstellten Testfälle berechnet wird:

$$COV = \frac{TC_{GPT-4o}}{TC_{Manual}} \times 100\%$$

Dabei gilt:

- TC_{GPT-4o} ist die Anzahl der von GPT-4o generierten Testfälle pro Testszenario.
- TC_{Manual} ist die Anzahl der von Pavankumar21github (2022) manuell erstellten Testfälle desselben Testszenario, welche als Benchmark dienen.

Den Wert für die Abdeckung misst sich mithilfe der Metriken Testfallstruktur und Factual Correctness. Pro Testszenario, zum Beispiel im Testszenario Register Account, wird GPT-4o dazu aufgefordert, alle möglichen Testfälle dieses Testszenarios zu generieren. Diese werden dokumentiert und ausgewertet. Die Testfälle von GPT-4o müssen dabei die Testfallstruktur aufweisen und gleichzeitig nur Testschritte besitzen, die faktisch korrekt und aufeinander aufgebaut sind.

Dies wird überprüft, in dem untersucht wird, ob die Informationen aus den Testfällen von GPT-4o mit denen aus den manuellen Testfällen und auf der OpenCart (o. D.) Webseite übereinstimmen. Die Anzahl der Testfälle, die durch GPT-4o pro Testszenario erstellt wurden und dabei beide Qualitätskriterien erfüllen ist der Wert TC_{GPT-4o} .

Die Testfälle von Pavankumar21github (2022), welche es pro Testszenario gibt, ist der Wert für TC_{Manual} . Wenn GPT-4o 20 Testfälle für ein Testszenario generiert hat, dabei aber zehn Testfälle entweder die Testfallstruktur nicht aufweisen oder nicht korrekte Testschritte besitzen und Pavankumar21github (2022) ebenfalls 20 Testfälle für dieses Testszenario erstellt hat, ist der Wert der Abdeckung in diesem Testszenario bei 50 %.

Die Testdaten, werden von GPT-4o in eine Excel Datei eingetragen und dann nach den drei oben genannten Qualitätskriterien ausgewertet. In Kapitel 5 finden sich die Ergebnisse. Eine Diskussion über die Ergebnisse und welche Schlüsse daraus gezogen werden können, findet sich in Kapitel 6.

Um die Auswertung der Ergebnisse und die Bedeutung der drei Qualitätskriterien zu verdeutlichen hilft folgende Tabelle: Sie zeigt einen Testfall mit einer negativen Testfallstruktur, einen mit einer Factual Correctness unter dem Wert von eins und einen Testfall, welcher eine positive Testfallstruktur aufweist und dabei eine Factual Correctness im Wert von eins besitzt:

Test Case ID	Test Case Title	Precondition	Test Scenario	Steps to Execute	Expected Result	Testfallstruktur eingehalten?	Factual Correctness?	Abdeckung
	Enter Valid Details and Register	Open OpenCart.com	Successful Registration	1. Click on 'Register' button 2. Fill all fields with valid data 3. Click 'Register' 4. User is redirected to confirmation page 5. Click 'Continue'		Nein! Test Case ID fehlt! Expected Result fehlt!	TP = 5 FP = 0 TN = 0 P = 1 R = 1 F1 = 1	Nein Testfallstruktur nicht eingehalten
TC002	Enter Valid Details and Register	Open OpenCart.com	Successful Registration	1. Click on 'Register' button 2. Click 'Continue' 3. 'Click Order' 4. User is redirected to confirmation page 5. Fill all fields with valid data	Welcome to OpenCart, your account has been created	Ja	TP = 1, 1. richtig FP = 4 2. Falsche Reihenfolge 3. Halluzination: Order Button gibt es nicht 4. Falsche Reihenfolge, erfolgreiche Weiterleitung ohne Felder auszufüllen 5. falsche Reihenfolge TN = 0 P = 1/5 = 0,20 R = 1/5 = 0,20 F1 = 0,20	Nein F1 ist nicht bei 1
TC003	Enter Valid Details and Register	Open OpenCart.com	Successful Registration	1. Click on 'Register' button 2. Fill all fields with valid data 3. Click 'Register' 4. User is redirected to confirmation page 5. Click 'Continue'	Welcome to OpenCart, your account has been created, verification email is sent	Ja	TP = 5 FP = 0 TN = 0 P = 1 R = 1 F1 = 1	Ja

Abbildung 4.5: Vergleich dreier Testfälle, die eine unterschiedliche Qualität der Qualitätskriterien aufweisen.

Der erste Testfall besitzt zwar eine Factual Correctness, jedoch eine negative Testfallstruktur. Da dieser keine ID besitzt, ist es schwierig diesen wiederzufinden. Auch wird nicht klar, was das Ziel dieses Testfalles ist, da es kein erwartetes Ergebnis gibt. Weil die Struktur fehlt, gilt dieser nicht als ein Testfall, um die Abdeckung zu berechnen.

Der zweite Testfall besitzt im Gegensatz dazu eine als positiv eingestufte Testfallstruktur. Allerdings ist die Factual Correctness nicht ausreichend. Im Testszenario Register testet dieser den Button „Order“, welchen es in diesem Testszenario nicht gibt. Es handelt sich um eine Halluzination. Außerdem ist die Reihenfolge, bis auf den ersten Testschritt, nicht richtig. Da die Factual Correctness nicht bei eins ist, wird dieser Testfall ebenfalls nicht berücksichtigt, um die Abdeckung zu berechnen.

Im Gegensatz zu dem ersten und zweiten Testfall besitzt der dritte Testfall sowohl eine positive Testfallstruktur als auch eine Factual Correctness von eins. Dadurch wird dieser Testfall für die Berechnung der Abdeckung berücksichtigt. Wenn in diesem Beispiel Pavankumar21github (2022) drei Testfälle geschrieben hat, wäre die Abdeckung der von GPT-4o generierten Testfälle bei 1/3. Da zwei der drei Testfälle nicht berücksichtigt wurden.

Dieser Vergleich zeigt anschaulich, wie wichtig die Einhaltung der drei Qualitätskriterien ist und wie diese ausgewertet werden.

5 Ergebnisse

Im Folgenden wurden die Ergebnisse, welche die von GPT-4o generierten Testfälle in Bezug auf die in Kapitel 4.4 beschriebenen Qualitätskriterien erreicht haben, ausgewertet und in diesem Kapitel veranschaulicht. Alle Testdaten, die Anforderungsspezifikationen, die manuellen sowie die von GPT-4 generierten Testfälle und die Auswertungstabellen finden sich im GitHub-Projekt von Müller (2025).

In dieser Arbeit wurden zehn Testszenarien mit einer unterschiedlichen Anzahl an Testfällen getestet. Dabei wurden von GPT-4o insgesamt 128 Testfälle generiert.

1. Testfallstruktur

Folgende Tabelle zeigt das Abschneiden der von GPT-4o erstellten Testfälle in Bezug auf das Qualitätskriterium „Testfallstruktur“, aufgelistet in zehn Testszenarien und der Gesamtzahl:

Testfallstruktur:

Testszenario:	Anzahl Testfälle:	1. Positiv:	2. Negativ:	Positiv in Prozent:
Register:	20	20	0	100 %
Login:	22	22	0	100 %
Logout:	10	10	0	100 %
Forgot Password:	22	22	0	100 %
Add to Cart:	9	9	0	100 %
My Account:	11	11	0	100 %
My Account Information:	10	10	0	100 %
Change Password:	10	10	0	100 %
Order History:	11	11	0	100 %
Currencies:	3	3	0	100 %
Gesamt:	128	128	0	100 %

Die von GPT-4o erstellten Testfälle, haben über alle zehn Testszenarien hinweg und in allen 128 Testfällen die in Kapitel 4.4.1 beschriebene Testfallstruktur eingehalten. Dabei hatten alle von GPT-4o generierten Testfälle eine ID, einen Namen, Vorbedingungen, ein Szenario als Beschreibung, Testschritte die nummeriert sind und ein erwartetes Ergebnis.

2. Factual Correctness

Folgende Tabelle zeigt das Abschneiden der von GPT-4o erstellten Testfälle in Bezug auf das Qualitätskriterium „Factual Correctness“ in zehn Testszenarien:

Factual Correctness:

Testszenario:	Anzahl Testfälle:	1. Positiv:	2. Negativ:	Positiv in Prozent:
Register:	20	17	3	85 %
Login:	22	17	5	77,3 %
Logout:	10	9	1	90 %
Forgot Password:	22	17	5	77,3 %
Add to Cart:	9	8	1	88,9 %
My Account:	11	6	5	54,5 %
My Account Information:	10	8	2	80 %
Change Password:	10	7	3	70 %
Order History:	11	8	3	72,7 %
Currencies:	3	3	0	100 %
Gesamt:	128	100	28	78,1 %

Die von GPT-4o generierten Testfälle wurden wie in Kapitel 4.4.2 beschrieben, auf ihre Korrektheit überprüft und ausgewertet. Dabei sind 100 Testfälle als positiv und 28 Testfälle als negativ eingestuft worden. Dies ergibt eine Positivrate von 78,1 %, aller von GPT-4o generierten Testfälle. Die Reichweite der Positivrate der einzelnen Testszenarien unterscheidet sich jedoch erheblich. So ist die niedrigste Positivrate im Testszenario „My Account“ erzielt worden, mit einem Wert von 54,5 %. Mit einem Wert von 100 % schneidet das Testszenario „Currencies“ ab.

Folgende Grafik zeigt anschaulich wie die Factual Correctness pro Testszenario abschnidet:

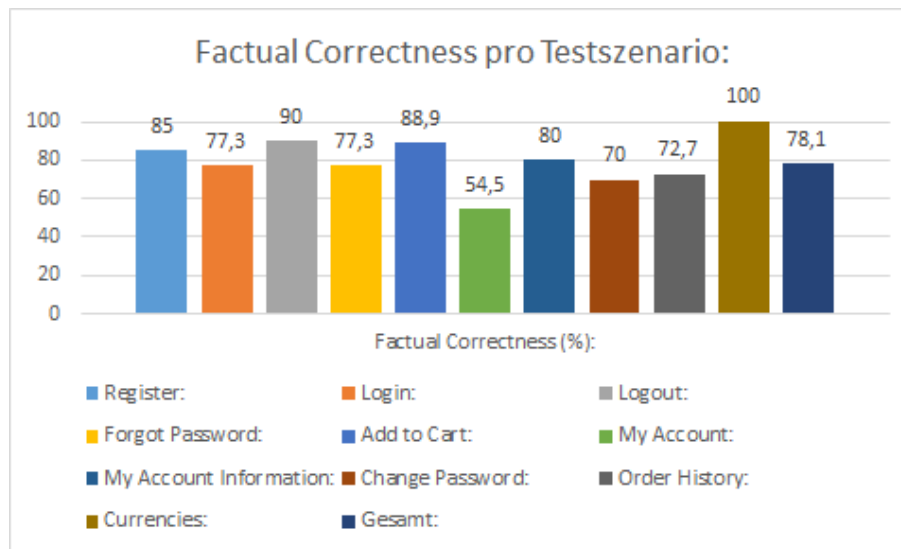


Abbildung 5.1: Faktuell Correctness pro Testszenario

Anhand dieser Grafik ist zu erkennen, dass die Factual Correctness pro Testszenario Schwankungen enthält, diese jedoch nie unter 50 % fällt.

3. Coverage (Deutsch: Abdeckung)

Folgende Tabelle zeigt das Abschneiden der von GPT-4o erstellten Testfälle in Bezug auf das Qualitätskriterium „Abdeckung“ in zehn Testszenarien: Dabei werden nur die Testfälle berücksichtigt, welche sowohl im Qualitätskriterium „Testfallstruktur“, als auch im Qualitätskriterium „Factual Correctness“ im positiven Bereich liegen:

Abdeckung:

Testszenario:	Anzahl manuelle Testfälle:	Anzahl GPT-4o Testfälle:	Abdeckung in Prozent:
Register:	27	17	63 %
Login:	23	17	73,9 %
Logout:	11	9	81,8 %
Forgot Password:	25	17	68 %
Add to Cart:	9	8	88,9 %
My Account:	10	6	60 %
My Account Information:	13	8	61,5 %
Change Password:	13	7	53,8 %
Order History:	13	8	61,5 %
Currencies:	3	3	100 %
Gesamt:	147	100	68 %

Die von GPT-4o erstellten Testfälle werden, wie in Kapitel 4.4.3 beschrieben, mit der Anzahl der von Pavankumar21github (2022) erstellten Testfälle verglichen, um die Abdeckungsrate zu erhalten. Dabei gibt es in den zehn Testszenarien 147 manuelle von Pavankumar21github (2022) erstellte Testfälle und 100 von GPT-4o generierte und als positiv eingestufte Testfälle. Dies ergibt eine Abdeckungsrate von 68 %. Wie in dem Qualitätskriterium „Factual Correctness“, gibt es in dem Qualitätskriterium „Abdeckung“ ebenfalls Schwankungen zwischen den einzelnen Testszenarien. Die niedrigste Abdeckungsrate mit einem Wert von 53,8 % ist in dem Testszenario „Change Password“ erzielt worden. Das Testszenario „Currencies“ schneidet mit einer Abdeckungsrate von 100 % ab.

Folgende Grafik zeigt anschaulich wie die Abdeckung pro Testszenario abschneidet:

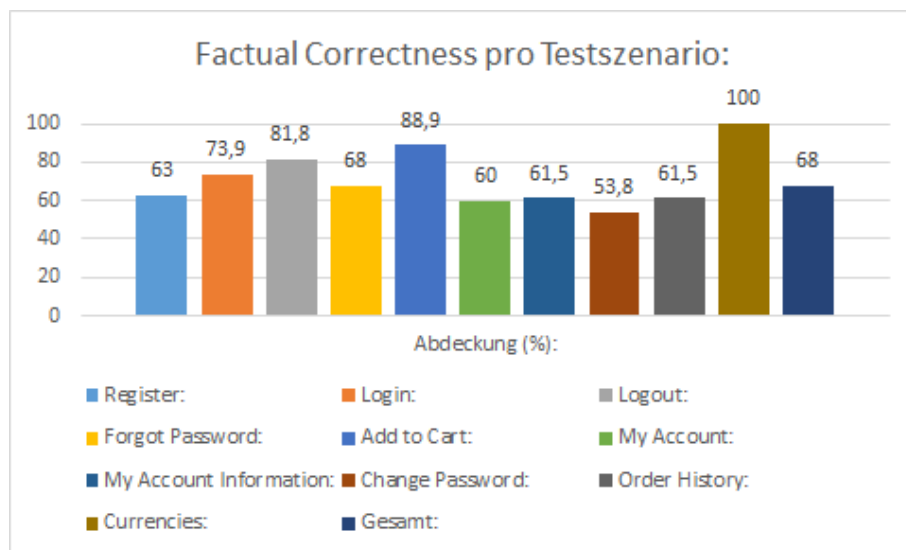


Abbildung 5.2: Abdeckung pro Testszenario

Anhand dieser Grafik ist zu erkennen, dass die Abdeckungsrate pro Testszenario Schwankungen enthält, diese jedoch nie unter 50 % fällt.

6 Diskussion

In dieser Arbeit wurden mit GPT-4o von OpenAI (o. D.) Anwendungstestfälle generiert. Dabei wurden diese anhand der in Kapitel 4.4 beschriebenen Qualitätskriterien ausgewertet. In diesem Kapitel werden die Ergebnisse nach Qualitätskriterium eingeordnet.

1. Testfallstruktur:

GPT-4o erreichte bei der Testfallstruktur einen Wert von 100 %. Dies zeigt, dass GPT-4o dazu in der Lage ist, die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) abzubilden. Somit können die von GPT-4o generierten Testfälle dokumentiert werden. Zukünftige Werke können darauf aufbauen und die Anpassungsfähigkeit von GPT-4o an individuellen Testfallstrukturen untersuchen. Anhand der aufgestellten Methodik lässt sich analysieren, ob GPT-4o in der Lage ist, die Testfallstruktur individuell anzupassen.

2. Factual Correctness:

Die Factual Correctness erreicht den Durchschnittswert von 78,1 %, wobei der niedrigste Wert 54,5 % und der höchste Wert bei 100 % liegt. Die restlichen acht Testszenarien liegen im Bereich zwischen 70 % und 90 %. Der Grund der Schwankungen kann an folgenden Gründen liegen. Wie Li et al. (2024) in ihrem Werk beschrieben, gibt es das Problem der Halluzination. So könnte GPT-4o in unterschiedlichen Testszenarien unterschiedlich stark halluziniert haben. Wie Iqbal und Qureshi (2022) in ihrem Werk erläutern, können Sprachmodelle bei längeren Texten Schwierigkeiten bekommen, die natürliche Sprache vollständig zu begreifen. Somit könnte in diesem Fall die Komplexität des Testszenarios eine Rolle dabei spielen, wie GPT-4o das Testszenario verstehen und passende Testfälle generieren kann. In dieser Arbeit hat die Anzahl der Testfälle die generiert wurden jedoch keine Auswirkung, auf den Wert der Factual Correctness des Testszenarios. Die Testszenarien „Register“, „Login“ und „Forgot Password“, mit 20 oder mehr generierten Testfällen schneiden in ihrer Positivrate besser ab als zum Beispiel „My Account“ oder „Change Password“.

3. Coverage (Deutsch: Abdeckung)

Die zehn von GPT-4o erstellten Testfälle erreichten bei der Abdeckungsrate einen Durchschnittswert von 68 %. Der niedrigste Wert eines Testszenario liegt bei 53,8 % und der höchste Wert bei 100 %. Die restlichen acht Testszenarien liegen im Bereich 60 % bis 88,9 %.

Die Schwankungen lassen sich dabei folgendermaßen erklären:

Da für Messung der Abdeckungsrate nur Testfälle berücksichtigt wurden, welche in den beiden Qualitätskriterien „Testfallstruktur“ und „Factual Correctness“ im positiven Bereich lagen, ist die Abdeckungsrate davon betroffen, wie sehr diese beiden Qualitätskriterien schwanken.

Ein weiterer Punkt, welcher berücksichtigt werden muss, ist die Benchmark. Für diese Arbeit wurden die von Pavankumar21github (2022) erstellten Testfälle als Benchmark verwendet, um die Abdeckungsrate zu bestimmen. Dieses Projekt von Pavankumar21github (2022) wurde jedoch im Jahr 2022 erstellt und könnte eine ältere Version von OpenCart (o. D.) getestet haben, in welcher es eine andere Anzahl an Funktionen pro Testszenario gegeben haben könnte. Auch ist nicht sicher ob diese selbst eine 100 % Abdeckungsrate erreichen, oder eventuell redundante Testfälle durchgeführt haben. Um die erste Anmerkung zu berücksichtigen, wurde bei der Auswertung der von GPT-4o erstellten Testfälle, diese mit den Informationen auf der OpenCart (o. D.) überprüft. Der zweite Punkt kann nicht ausgeräumt werden. Weil es jedoch wichtig ist, einen Vergleichswert zu haben, kann es dennoch benutzt werden.

7 Fazit und Ausblick

Diese Arbeit untersucht den Einsatz von Künstlicher Intelligenz zur Testfallgenerierung. Ziel dieser Arbeit ist es, folgende Forschungsfrage zu beantworten:

Ist es möglich, dass das LLM GPT-4o von OpenAI (o. D.) dazu in der Lage ist, aus Anforderungsspezifikationen und Screenshots mit Website-Informationen, Anforderungstestfälle zu erzeugen, die in Bezug auf messbare Kriterien für den praktischen Einsatz in der Testfallerstellung geeignet sind?

Um diese Frage zu beantworten, wurden von GPT-4o Anwendungstestfälle generiert. Dabei wurden zehn unterschiedliche Testszenarien mit insgesamt 128 Anwendungsfällen getestet. Diese Testfälle wurden dann ausgewertet.

Aufgrund der vorliegenden Ergebnisse kommt diese Arbeit zu folgenden Schluss:
Es ist möglich, mit dem LLM GPT-4o Anforderungstestfälle zu erzeugen, welche in Bezug auf ihrer Abdeckung, Factual Correctness und Testfallstruktur nutzbar sind.

Die Testfallstruktur hat in allen Testszenarien hindurch, einen Wert von 100 % erreicht. Dies zeigt, dass GPT-4o dazu in der Lage ist, Testfälle zu generieren, welche die gewünschte Testfallstruktur aufweisen.

Bei den beiden Qualitätskriterien „Factual Correctness“ und „Abdeckung“ gab es jedoch Schwankungen pro Testszenario. Da in diesen beiden Qualitätskriterien der Wert von Testszenario zu Testszenario unterschiedlich ist, kann zu diesen beiden keine allgemein gültige Aussage getroffen werden kann.

Dennoch konnten die von GPT-4o generierten Testfälle über alle Testszenarien hinweg einen Wert von über 50 % in Bezug auf ihre Abdeckung, Factual Correctness und Testfallstruktur aufweisen. Dadurch kann ein LLM wie GPT-4o ein nutzbares Werkzeug zur Erstellung von Testfällen sein. Es ist jedoch wichtig, diese zu überprüfen, da sie nicht automatisch richtig sein müssen und je nach Testszenario eine unterschiedliche Zuverlässigkeit haben.

Die Ergebnisse dieser Arbeit zeigen, dass der Einsatz von Künstlicher Intelligenz zur Testfallgenerierung eine praktische Bedeutung besitzt. Zukünftige Werke können dazu andere LLMs verwenden und Vergleiche ziehen und andere Anwendungen testen, um Optimierungsmöglichkeiten beim Einsatz von Künstlicher Intelligenz zur Testfallgenerierung zu untersuchen. Außerdem wird empfohlen, eine größere Vergleichsmenge an unterschiedlichen manuell erstellten Testfällen zu nehmen, um eine bessere Referenz für den Vergleich zu erhalten.

Literatur

- Backlinko. (2025). *ChatGPT / OpenAI Statistics: How Many People Use ChatGPT?* [abgerufen am 18.02.2025]. <https://backlinko.com/chatgpt-stats#chat-gpt-weekly-active-users-worldwide>
- Barmi, Z. A., Ebrahimi, A. H., & Feldt, R. (2011). Alignment of Requirements Specification and Testing: A Systematic Mapping Study. *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. <https://ieeexplore.ieee.org/document/5954452>
- Bhandari, J., Knechtel, J., Narayanaswamy, R., Garg, S., & Karri, R. (2024). LLM-Aided Testbench Generation and Bug Detection for Finite-State Machines. *Computer Science > Hardware Architecture*, 1. <https://arxiv.org/abs/2406.17132>
- Bhatt, C., Kumar, I., Vijayakumar, V., Singh, K. U., & Kumar, A. (2020). The state of the art of deep learning models in medical science and their challenges. *Multimedia Systems*, 27, 599–613. <https://link.springer.com/article/10.1007/s00530-020-00694-1>
- Bozic, S. (2022). *Künstliche Intelligenz im Testprozess* [abgerufen am 03.11.2024]. <https://bbv-software.de/insights/blog/ai-testing/>
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., & Xie, X. (2023). A Survey on Evaluation of Large Language Models. *Computer Science > Computation and Language*, 9. <https://arxiv.org/abs/2307.03109>
- Dymatrix. (2018). *Deep-Learning im Digital Business* [abgerufen am 13.01.2025]. <https://www.dymatrix.de/dymatrixblog/deep-learning-definition>
- Enderli, D. (2019). *Testfalldesign für SAP Applikationen* [abgerufen am 16.01.2025]. <https://community.sap.com/t5/technology-blogs-by-members/testfalldesign-f%C3%BCr-sap-applikationen/ba-p/13451295>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *Computer Science > Computation and Language*, 5. <https://arxiv.org/abs/2312.10997>
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition* [ISBN: 978-1-4920-3264-9]. O'Reilly Media, Inc.
- Gu, J., Han, Z., Chen, S., Beirami, A., He, B., Zhang, G., Liao, R., Qin, Y., Tresp, V., & Torr, P. (2023). A Systematic Survey of Prompt Engineering on Vision-Language Foundation Models. *Computer Science > Computer Vision and Pattern Recognition*, 1. <https://arxiv.org/abs/2307.12980>
- Gu, S., Zhang, Q., Fang, C., Tian, F., Zhu, L., Zhou, J., & Chen, Z. (2024). TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration. *Computer Science > Software Engineering*, 5. <https://arxiv.org/abs/2408.03095>
- Hecker, D., Döbel, I., Petersen, U., Rauschert, A., Schmitz, V., & Voss, A. (2018). Zukunftsmarkt Künstliche Intelligenz. <https://kmu-digital.eu/de/service-kompetenz/publikationen/dokumente/studien/198-zukunftsmarkt-kuenstliche-intelligenz-potenziale-und-anwendungen>

- Hodiak, V. (2024). *NLP vs LLM: Navigating Differences and Core Advantages* [abgerufen am 16.01.2025]. <https://otakoyi.software/blog/nlp-vs-llm-navigating-differences-and-core-advantages>
- Honroth, T., Siebert, J., & Kelbert, P. (2024). *Retrieval Augmented Generation (RAG): Chatten mit den eigenen Daten* [abgerufen am 13.01.2025]. <https://www.iese.fraunhofer.de/blog/retrieval-augmented-generation-rag/>
- Hugging Face. (o. D.). *The AI community building the future.* [abgerufen am 16.01.2025]. <https://huggingface.co/>
- Iqbal, T., & Qureshi, S. (2022). The survey: Text generation models in deep learning. *Journal of King Saud University – Computer and Information Sciences*, 34(6), 2515–2528. <https://doi.org/10.1016/j.jksuci.2020.04.001>
- ISO/IEC/IEEE 29119-1:2022-01. (2022). *Software and systems engineering — Software testing Part 1: General concepts* [abgerufen am 16.01.2025]. <https://www.iso.org/standard/81291.html>
- ISTQB/GTB Standardglossar der Testbegriffe. (2017). *Standardglossar der Testbegriffe* [abgerufen am 16.01.2025]. https://swisstestingboard.org/wp-content/uploads/2018/Documents/Glossar/Glossar%20Deutsch-Englisch%203_11.pdf
- Jama Software. (o. D.). *Functional vs. nonfunctional requirements: What's the difference?* [abgerufen am 16.01.2025]. <https://www.jamasoftware.com/requirements-management-guide/writing-requirements/functional-vs-non-functional-requirements>
- Kaddour, J., Harris, J., Mozes, M., Bradley, H., Raileanu, R., & McHardy, R. (2023). Challenges and Applications of Large Language Models. *Computer Science > Computation and Language*, 1. <https://arxiv.org/abs/2307.10169>
- Kerner, S. M. (2024). *Large Language Model (LLM)* [abgerufen am 03.11.2024]. <https://www.computerweekly.com/de/definition/Large-Language-Model-LLM>
- Khan, F. I., Mahmud, F. U., & Hoseen, A. (2024). A New Approach of Software Test Automation Using AI. *Yingyong Jichu yu Gongcheng Kexue Xuebao/Journal of Basic Science and Engineering*, 21(2), 559–570. https://www.researchgate.net/publication/380459206_A_NEW_APPROACH_OF_SOFTWARE_TEST_AUTOMATION_USING_AI
- Kirinuki, H., & Tanno, H. (2024). ChatGPT and Human Synergy in Black-Box Testing: A Comparative Analysis. *Computer Science > Software Engineering*, 1. <https://arxiv.org/abs/2401.13924>
- Lazić, L. (2013). Software Quality Testing Metrics. *Computer Science*. <https://www.semanticscholar.org/paper/Software-Quality-%26-Testing-Metrics-Lazic/2df2f394184e3620e747b738733b957aac46ad8d>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Computer Science > Computation and Language*, 4. <https://arxiv.org/abs/2005.11401>
- Li, J., Chen, J., Ren, R., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2024). The Dawn After the Dark: An Empirical Study on Factuality Hallucination in Large Language Models. *Computer Science > Computation and Language*, 1. <https://arxiv.org/abs/2401.03205>
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 55, 1–35. <https://dl.acm.org/doi/10.1145/3560815>

- Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., & Wang, Q. (2024). Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. *ICSE '24: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, (100), 1–13. <https://dl.acm.org/doi/10.1145/3597503.3639180>
- Miesle, P. (2023). *Retrieval-Augmented Generation (RAG) Explained: Understanding Key Concepts* [abgerufen am 13.01.2025]. <https://www.datastax.com/guides/what-is-retrieval-augmented-generation>
- Mocko, M., & Bitton-Bailey, A. (o. D.). *Areas of Artificial Intelligence* [abgerufen am 13.01.2025]. <https://ufl.pb.unizin.org/artificialintelligence/chapter/six-area-of-artificial-intelligence/>
- Möllers, N. (2024). *Künstliche Intelligenz und Datenschutz* [abgerufen am 03.11.2024]. <https://keyed.de/blog/kuenstliche-intelligenz-und-datenschutz/>
- Müller, M. (2025). *MrMueller15/Bachelor-Thesis* [abgerufen am 11.02.2025]. <https://github.com/MrMueller15/Bachelor-Thesis>
- Mustafa, A., Wan-Kadir, W. M. N., Ibrahim, N., Shah, M. A., Younas, M., Khan, A., Zareei, M., & Alanazi, F. (2021). Automated Test Case Generation from Requirements: A Systematic Literature Review. *Computers, Materials Continua*, 67, 1819–1833. <https://www.techscience.com/cmc/v67n2/41326>
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2024). A Comprehensive Overview of Large Language Models. *Computer Science > Computation and Language*, 10. <https://arxiv.org/abs/2307.06435>
- Nirpals, P. B., & Kale, K. (2011). A Brief Overview Of Software Testing Metrics. *International Journal on Computer Science and Engineering*, 3. https://www.researchgate.net/publication/50247283_A_Brief_Overview_Of_Software_Testing_Metrics
- OpenAI. (o. D.). *OpenAI* [abgerufen am 17.01.2025]. <https://openai.com/>
- OpenCart. (o. D.). *OpenCart* [abgerufen am 11.02.2025]. <https://www.opencart.com/>
- Ouédraogo, W. C., Kaboré, K., Tian, H., Song, Y., Koyuncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2024). Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. *Computer Science > Software Engineering*, 1(1). <https://arxiv.org/abs/2407.00225>
- Pavankumar21github. (2022). *Manual-Testing-Project* [abgerufen am 11.02.2025]. <https://github.com/Pavankumar21github/Manual-Testing-Project>
- Ragas. (2024). *Factual Correctness* [abgerufen am 21.02.2025]. https://docs.ragas.io/en/latest/concepts/metrics/available_metrics/factual_correctness/
- Salemi, A., & Zamani, H. (2024). Evaluating Retrieval Quality in Retrieval-Augmented Generation. *SIGIR '24: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2395–2400. <https://dl.acm.org/doi/10.1145/3626772.3657957>
- Schmid, A., & Sommer, M. (2024). *Retrieval Augmented Generation – RAG: Generative künstliche Intelligenz-Lösungen mit Unternehmensdaten anreichern* [abgerufen am 13.01.2025]. <https://aracom.de/retrieval-augmented-generation/>
- Siebert, J. (2024). *Halluzinationen von generativer KI und großen Sprachmodellen (LLMs)* [abgerufen am 22.11.2024]. <https://www.iese.fraunhofer.de/blog/halluzinationen-generative-ki-llm/>

- Statistisches Bundesamt. (2024). *Jedes fünfte Unternehmen nutzt künstliche Intelligenz* [abgerufen am 05.11.2024]. https://www.destatis.de/DE/Presse/Pressemitteilungen/2024/11/PD24_444_52911.html
- Streim, A., & Hecker, J. (2023). *Deutsche Wirtschaft drückt bei Künstlicher Intelligenz aufs Tempo* [abgerufen am 03.11.2024]. <https://www.bitkom.org/Presse/Presseinformation/Deutsche-Wirtschaft-drueckt-bei-Kuenstlicher-Intelligenz-aufs-Tempo>
- Tran, H. K. V., b[in] A[li], N., Unterkalmsteiner, M., Börstler, J., & Chatzipetrou, P. (2025). Quality attributes of test cases and test suites – importance challenges from practitioners’ perspectives. *Software Quality Journal*, 33. <https://link.springer.com/article/10.1007/s11219-024-09698-w>
- Visure. (o. D.). *Was ist Anforderungsspezifikation: Definition, beste Tools und Techniken* [abgerufen am 15.01.2025]. <https://visuresolutions.com/de/blog/requirements-specification/>
- Weingartz, R., & Suleymanov, N. (2024). *Wie man bessere Tests mit KI schreibt* [abgerufen am 03.11.2024]. <https://aqua-cloud.io/de/ai-to-write-tests/>
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T., Guan, N., & Xue, C. J. (2024). Retrieval-Augmented Generation for Natural Language Processing: A Survey. *Computer Science > Computation and Language*, 2. <https://arxiv.org/abs/2407.13193>
- Yuan, W., Neubig, G., & Liu, P. (2021). BARTSCORE: Evaluating Generated Text as Text Generation. *Computer Science > Computation and Language*, 2. <https://arxiv.org/abs/2106.11520>
- Zhu, L., Spachos, P., Pensini, E., & Plataniotis, K. (2021). Deep learning and machine vision for food processing: A survey. *Current Research in Food Science*, 4, 233–249. <https://www.sciencedirect.com/science/article/pii/S2665927121000228?via%3Dihub>

Eidesstattliche Erklärung

Hiermit erkläre ich eidesstattlich, dass die vorliegende Arbeit von mir selbstständig und ohne unerlaubte Hilfe angefertigt wurde, insbesondere, dass ich alle Stellen, die wörtlich oder annähernd wörtlich oder dem Gedanken nach aus Veröffentlichungen und unveröffentlichten Unterlagen und Gesprächen entnommen worden sind, als solche an den entsprechenden Stellen innerhalb der Arbeit durch Zitate kenntlich gemacht habe, wobei in den Zitaten jeweils der Umfang der entnommenen Originalzitate kenntlich gemacht wurde. Ich bin mir bewusst, dass eine falsche Versicherung rechtliche Folgen haben wird.

Ort, Datum

Unterschrift

Anhang

Alle Testdaten sind über folgende Links zu finden:

Projekt von Pavankumar21github (2022) von welchem die manuellen Testfälle stammen:
<https://github.com/Pavankumar21github/Manual-Testing-Project>

Projekt von Müller (2025) in welchem alle von GPT-4o von OpenAI (o. D.) erstellten Testfälle zu finden sind: <https://github.com/MrMueller15/Bachelor-Thesis>

Die Webseite OpenCart (o. D.) deren Funktionen von Pavankumar21github (2022) und von GPT-4o von OpenAI (o. D.) und getestet wurden: <https://www.opencart.com/>